Computers & Security 138 (2024) 103658

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

# Improved privacy-preserving PCA using optimized homomorphic matrix multiplication

Xirong Ma [a], Chuan Ma [b], Yali Jiang [a,*], Chunpeng Ge [a,c]

[a] School of Software, Shandong University, China
[b] Zhejiang Lab, China
[c] Joint SDU-NTU Centre for Artificial Intelligence Research (C-FAIR), China

ARTICLE INFO

*Keywords:*
Privacy-preserving PCA
CKKS
Homomrophic matrix multiplication
Cloud computing
Machine learning as a service

ABSTRACT

Principal Component Analysis (PCA) is a pivotal technique widely utilized in the realms of machine learning and data analysis. It aims to reduce the dimensionality of a dataset while minimizing the loss of information. In recent years, there have been endeavors to utilize homomorphic encryption in privacy-preserving PCA algorithms for secure cloud computing. These approaches commonly employ a PCA routine known as PowerMethod, which takes the covariance matrix as input and generates an approximate eigenvector corresponding to the primary component of the dataset. However, their performance is constrained by the absence of an efficient homomorphic covariance matrix computation circuit and an accurate homomorphic vector normalization strategy in the PowerMethod algorithm. In this study, we propose a novel approach to privacy-preserving PCA that addresses these limitations, resulting in superior efficiency, accuracy, and scalability compared to previous approaches.

We attain such efficiency and precision through the following contributions: (i) We implement space and speed optimization techniques for a homomorphic matrix multiplication method, specifically tailored for parallel computing scenarios. (ii) Leveraging the benefits of this optimized matrix multiplication, we devise an efficient homomorphic circuit for computing the covariance matrix homomorphically. (iii) Utilizing the covariance matrix, we develop a novel and efficient homomorphic circuit for the PowerMethod that incorporates a universal homomorphic vector normalization strategy to enhance both its accuracy and practicality.

Our privacy-preserving PCA scheme, implemented using our innovative homomorphic PowerMethod circuit, surpasses the state-of-the-art approach with an average speedup of 1.9 times on datasets with size $200 \times 256$. Notably, our scheme demonstrates an even more remarkable estimated speedup of 25 times when applied to larger datasets of size $60000 \times 256$, along with an R2 accuracy improvement of up to 0.285, showcasing efficiency and accuracy that has not been reported by previous approaches.

## 1. Introduction

Principal Component Analysis (PCA) (Hotelling, 1933), (Pearson, 1901) is a widely employed dimensionality reduction technique. It enables high-dimensional data mapped to a lower-dimensional space, preserving the most essential features while minimizing redundant information and noise. In essence, when applied to a dataset, PCA aims to identify a set of vectors known as principal components which represent the directions of maximum variance in the dataset. Subsequently, it becomes possible to project the dataset onto these principal components to achieve a lower-dimensional representation. PCA is widely adopted in data analysis and machine learning, including data preprocessing, feature extraction, data compression, and visualization. It helps us understand the relationships within a dataset and provides a simpler and more manageable representation.

Due to the significance and versatility of PCA, it is leveraged as one of the supported technologies in cloud computing services, enabling users to perform enhanced data analysis leveraging the computational power of the cloud. However, concerns regarding the trustworthiness of cloud storage hinder users from directly performing analysis and processing of sensitive data in the cloud. Instead, it is needed to incorporate privacy protection measures into their data before outsourcing it to mitigate the risk of potential attacks from an untrusted cloud environment. Specifically, users and cloud servers can negotiate and

* Corresponding author.
*E-mail address:* jiang.yl@sdu.edu.cn (Y. Jiang).

https://doi.org/10.1016/j.cose.2023.103658
Received 5 July 2023; Received in revised form 24 October 2023; Accepted 12 December 2023
Available online 15 December 2023

employ privacy-preserving algorithm routines to process sensitive data in the cloud while ensuring privacy. Therefore, the proposition of a practical and efficient privacy-preserving PCA technique would greatly benefit users by enabling them to perform PCA on sensitive data in the cloud.

To provide readers with a more comprehensive understanding of privacy-preserving PCA in the context of cloud computing, we illustrate the concept with a simple scenario: A resource-constrained user wishes to perform machine learning tasks using the platform and computational resources of a cloud server, where PCA may emerge as a pivotal component in the initial phase of model training, particularly when confronted with datasets of substantial complexity, such as those containing myriad attributes like images. However, the dataset in question is imbued with a sense of privacy, and the user is reluctant to disclose the privacy to the cloud. In this predicament, the conventional approach of executing PCA on the cloud becomes untenable, given the inherent inability to outsource confidential data. Consequently, a meticulously crafted privacy-preserving PCA solution should be devised, ensuring that the user can execute PCA seamlessly on the cloud, all the while guarding the privacy of the dataset.

It is imperative to note that in this paper, our primary emphasis centers around the algorithms allowing the user to privately obtain the principal components from his or her private datasets leveraging the computational resources of the cloud, based on the aforementioned cloud computing scenario.

Nevertheless, privacy-preserving PCA can have different meanings in other contexts or scenarios. In some research, privacy-preserving PCA is considered the process of using pre-computed principal components to conduct the dimensionality reduction mapping of a private dataset (Arnold and Saniie, 2023). This differs from our focus, and the distinction can be likened to the machine learning terms “training” (principal components searching) and “inference” (dataset projecting).

In other cases, researchers consider the scenario of multi-party privacy-preserving PCA, where multiple entities each holding a private dataset aim to compute the PCA results of the merged dataset without disclosing individual privacy (Liu et al., 2020), (Froelicher et al., 2023). In this situation, the major goal is to distribute computation tasks among multiple parties and preserve privacy for each data owner, where a cloud server is not necessarily needed, and privacy-preserving PCA may be achieved in a distributed manner among multiple parties.

One possible approach for implementing a privacy-preserving PCA technique in our cloud computing scenario involves utilizing homomorphic encryption. Homomorphic encryption is an important privacy-preserving technique that allows computations to be performed on encrypted data without decryption. This technology serves to safeguard data privacy while preserving data usability, rendering it extensively employed in domains such as cloud computing and cross-domain computation. Over the past fifteen years, a class of homomorphic schemes based on the Ring Learning With Errors (R-LWE) problem (Lyubashevsky et al., 2010) has rapidly developed (e.g., but not limited to Brakerski et al., 2014, Fan and Vercauteren, 2012, Cheon et al., 2017, Cheon et al., 2019). These schemes naturally possess SIMD properties and support homomorphic addition and multiplication operations. As a result, a plethora of privacy-preserving data analysis algorithms based on these schemes have emerged, including privacy-preserving PCA schemes tailored for cloud service scenarios (Lu et al., 2016), (Rathee et al., 2018), (Panda, 2021). These existing homomorphic encryption-based privacy-preserving PCA methods utilized an iterative algorithm known as PowerMethod to compute the dominant eigenvector of the covariance matrix of the dataset, which corresponds to the first principal component. This algorithm selects an initial approximation of the dominant eigenvector and continually applies the covariance matrix transformation to refine its approximation (see Algorithm 1 for detail). There are two major limitations among the homomorphic PoweMethod algorithms in the previous approaches.

Firstly, these approaches lack the capability to compute the covariance matrix homomorphically, the PowerMethod necessitates the input of the dataset covariance matrix, but to our knowledge, previous schemes have not offered a homomorphic solution for computing this matrix. Instead, they resort to alternative methods. Some require users to compute the covariance matrix locally which burdens the users with additional computational tasks and deviates from the original intent of cloud services (Rathee et al., 2018). Others decompose the covariance matrix transformation within the PowerMethod into dataset matrix transformations to avoid explicit involvement of the covariance matrix but introduce extra computational complexity (Panda, 2021).

Secondly, there is a potential loss of accuracy due to the absence of a universal vector normalization strategy. In each iteration of PowerMethod, normalization is required to control the length of the vector. In the homomorphic context, iterative algorithms are typically used to approximate the inverse square root function for normalization. The accuracy of these algorithms heavily relies on the selection of parameters such as the evaluation interval and the number of iterations. To our knowledge, no prior research has presented a universal strategy to determine these parameter settings in the realm of homomorphic PowerMethod algorithms. Consequently, existing homomorphic PowerMethod algorithms may potentially suffer from inaccuracy due to this inherent limitation.

### 1.1. Our contributions

We propose an efficient privacy-preserving PCA algorithm that overcomes the obstacles above. To achieve this, we make the following contributions.

**Homomorphic Covariance Matrix Computation with Optimized Matrix Multiplication (Section 4, 5):** We enhance the efficiency of a cutting-edge homomorphic matrix multiplication algorithm, both in terms of time and space, to render it more apt for parallel computing scenarios. Subsequently, we employ it as a fundamental element to design a proficient algorithm for homomorphic covariance matrix computation, which harnesses the parallel computation capabilities of multiple matrix multiplication instances.

**Privacy-preserving PCA using Homomorphic PowerMethod (Section 6):** PowerMethod consists of two main components: covariance matrix transformation and vector normalization. We first design an efficient homomorphic circuit for covariance matrix transformation. Next, we employ an iterative algorithm approximating the inverse square root (InvSRT) function to perform the vector normalization and propose a systematic approach to parameterize the iterative algorithm. Finally, we present the process of performing privacy-preserving PCA utilizing our homomorphic PowerMethod circuit.

**Implementation (Section 7)** We compare our proposed solution with existing approaches, highlighting that our algorithm outperforms previous methods in terms of computational efficiency, precision, and scalability.

## 2. Related work

### 2.1. Privacy-preserving PCA in cloud computing scenario

The initial attempts at performing PCA on an RLWE-based homomorphic encryption scheme are made by Lu et al. (2016) and Rathee et al. (2018). They propose using an iterative algorithm called PowerMethod to compute principal components in a homomorphic encryption setting. The PowerMethod iteratively computes $\mathbf{v} \leftarrow \mathrm{Cov} \cdot \mathbf{v}$, and it can be proven that $\mathbf{v}$ converges to the dominant eigenvector of the covariance matrix Cov after a finite number of steps, where the dominant eigenvector is equivalent to a principal component. However, they find the homomorphic computation of the covariance matrix to be inefficient for arbitrary sample sizes. Therefore, their scheme requires the client (of cloud service) to compute and send the encryption of $\sum \mathbf{x}_i^T \mathbf{x}_i$

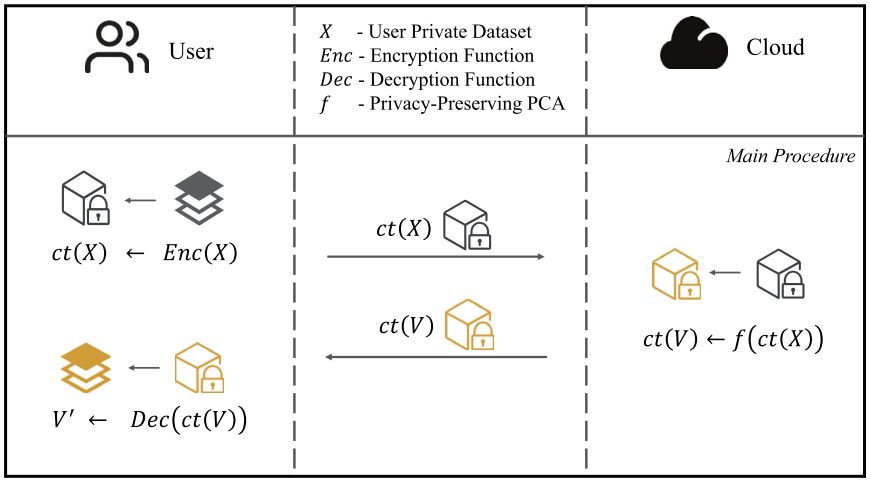


**Fig. 1.** System architecture.

(where $\mathbf{x}_i$ represents the row vector form of the samples in the dataset $X$), which not only causes the computational burden on the user but also becomes trivial because PowerMethod is no harder than computing $\sum \mathbf{x}_i^T \mathbf{x}_i$ in plaintext space. Furthermore, their PCA schemes cannot normalize the approximate eigenvector $\mathbf{v}$ during the iterations of PowerMethod, and the length of the approximate eigenvector is prone to overflow since the homomorphic encryption they used only supports homomorphic operations on modular integers (Brakerski et al., 2014).

Subsequently, a work by Panda (Panda, 2021) migrates the PowerMethod to another RLWE-based homomorphic encryption scheme known as CKKS that supports approximate computation on complex numbers (Cheon et al., 2017). The CKKS scheme enables them to perform vector normalization using an iterative algorithm of the inverse square-root function. However, their work provides only one possible implementation of the iterative algorithm without demonstrating how to choose its evaluation interval and the number of iterations. As a result, their solution lacks both accuracy and practicality. Furthermore, the PowerMethod in their work is done by iteratively computing $\mathbf{v} \leftarrow 1/N \cdot X^T(X\mathbf{v})$ (where $X$ is the centered dataset), rather than directly computing the covariance matrix transformation. This makes the complexity of the PowerMethod dependent on the size of the dataset rather than the size of the covariance matrix, which is inefficient for datasets with a large number of samples.

### 2.2. Homomorphic encryption matrix multiplication

Numerous approaches are proposed on how to construct secure matrix multiplication algorithms using homomorphic encryption schemes based on the RLWE problem (Duong et al., 2016), (Mishra et al., 2021), (Rathee et al., 2018), (Jiang et al., 2018), (Wang and Huang, 2019). Among them, the work by Jiang et al. (2018) introduces a scheme for secure real number matrix multiplication based on the CKKS scheme. To the best of our knowledge, it remains the state-of-the-art RLWE homomorphic encryption-based approach for real number matrix multiplication. In a recent study by Jang et al. (2022), they further improve this scheme by migrating it to a variant of CKKS based on the multivariate polynomial learning with errors (m-RLWE) problem.

Nevertheless, few have systematically discussed the performance of homomorphic matrix multiplication within parallel computation scenarios. We believe that delving into this matter is of utmost importance, as parallel computation plays a pivotal role in expediting extensive matrix operations that involve multiple ciphertexts. Efficiently executing multiple instances of homomorphic matrix multiplication in parallel is instrumental in accelerating large-scale computations, warranting a thorough exploration of this aspect.

## 3. Preliminaries

### 3.1. General notation

We use italic letters, such as $a$, to represent polynomial elements or numbers. We use bold letters, such as $\mathbf{a}$, to represent vectors, and uppercase letters, such as $A$, to represent matrices. The symbols $\oplus$ and $\odot$ denote elementwise addition and multiplication. The notation $\rho(\mathbf{a}; k)$ represents the result of cyclically left-shifting (or rotating) the components of vector $\mathbf{a}$ by $k$ positions. Additionally, we use $(\mathbb{Z}/q\mathbb{Z}, +)$ to represent a modulo $q$ additive group, where we use two sets of integers: $(-q/2, q/2]$ and $[0, q-1]$ to refer to two different representations of the residue class. We define $[a]_q$ as the result of reducing $a$ modulo $q$ that falls into $(-q/2, q/2]$, and $a \mod q$ as the result falling into $[0, q-1]$.

### 3.2. System architecture

The system architecture for privacy-preserving PCA in the cloud computing context comprises two entities: the user and the cloud server engaged in a privacy-preserving protocol (see Fig. 1). The user aims to upload his or her private dataset to the untrusted cloud, leveraging its computational power for PCA and obtaining the computed result. During this procedure, the user encrypts the private dataset $X$ using some encryption function $Enc$ resulting in ciphertext $ct(X)$ to prohibit the cloud from gaining any useful information related to user data. Then, after receiving $ct(X)$, the cloud performs a privacy-preserving PCA taking $ct(X)$ as input and outputting $ct(V)$ as the ciphertext encrypting the expected PCA result. Finally, the user can decrypt $ct(V)$ and recover the expected result by using the function $Dec$.

### 3.3. Threat model

We base our system architecture on a semi-honest security model. In specific terms, we consider the user's dataset to be private, while the privacy-preserving PCA algorithm executed in the cloud is visible to both the user and the cloud, making it non-private. Thus, the cloud is assumed to be semi-honest from the perspective of the user. This implies that the cloud will not actively violate the established interaction rules and protocols, ensuring the correct execution of privacy-preserving PCA. However, the cloud maintains a curiosity about the user's private data, attempting to access the underlying plaintexts of the received ciphertexts.

### 3.4. Homomorphic encryption for arithmetic of approximate numbers

Homomorphic encryption is a cutting-edge field in cryptography and serves as a powerful component for privacy-preserving computations.

The CKKS scheme we mentioned earlier is a homomorphic encryption scheme supporting approximate arithmetic on complex vector space. It provides an encoding method to store up to $N/2$ floating point values in plaintext or ciphertext, which are represented by polynomials in the domain $R_{Q_L} = \mathbb{Z}_{Q_L}[X]/(X^N + 1)$, where $Q_L = \prod_0^L q_i$, and $N$ is a power of 2. Operations over the domain are performed simultaneously on all encoded values, known as SIMD processing. The security of CKKS is based on the RLWE problem, so some noise is introduced into the encrypted values. We briefly introduce the operations provided by CKKS in the following.

- $s \leftarrow \text{SecKeyGen}(\cdot)$ generates the secret key $s$
- $pk, swks \leftarrow \text{PubKeyGen}(s)$ generates public key $pk$ and a series of switching keys $swks = \{swk_{s_i,s_j}\}$ where $swk_{s_i,s_j}$ helps switch the secret inside a ciphertext from $s_j$ to $s_i$. Notice that switching keys can be quite large in size. Typically, a group of switching keys together can occupy a space ranging from tens of megabytes to several gigabytes or even more.
- $ct(m) \in R^2_{Q_L} \leftarrow \text{Enc}(pk, m)$ encryptes a plaintext vector $p$.
- $m' \leftarrow \text{Dec}(sk, ct(m))$ decrypts the ciphertext $ct(m)$ and output $m' \approx m$
- $ct(m_1 + m_2) \leftarrow \text{Add}(ct(m_1), ct(m_2))$ performs addition of the encrypted vectors.
- $ct(m_1 \odot m_2) \leftarrow \text{PMult}(ct(m_1), m_2)$ performs elementwise multiplication between encrypted vector $ct(m_1)$ and plaintext vector $m_2$. Rescaling is needed to control the noise growth in the result.
- $ct(m_1 \odot m_2) \leftarrow \text{Mult}(ct(m_1), ct(m_2))$ performs elementwise multiplication between encrypted vectors. Rescaling and relinearization operations are required to control the size and noise growth of the result.
- $ct(\rho(m,k)) \leftarrow \text{Rot}(ct(m), rtk_k)$ performs cyclic rotation of step $k$ on the encrypted vector, where $rtk_k$ is an instance of switching key $swk_{s^{5^k},s}$ and $s$ is the secret inside the encrypted vector.

Notably, for a given set of modulus products $Q_L$, the homomorphic operations are limited to a multiplication depth of at most $L$ because rescaling consumes the modulus level of ciphertexts (any ciphertext in $R^2_{Q_l}$ is at the $l$-th modulus level, and rescaling brings the ciphertext to a lower level $R^2_{Q_{l-1}}$). Once the modulus level of a ciphertext reaches 0, it can no longer participate in homomorphic multiplication operations. To perform operations with a greater multiplication depth, we need to conduct an operation called modulus refresh which raises the modulus level of the ciphertext. The modulus refresh can be achieved either by decrypting and re-encrypting the ciphertexts or by applying the bootstrapping function to the ciphertexts.

We build our privacy-preserving PCA scheme upon a full Residue Number System (RNS) variant of the CKKS scheme (Cheon et al., 2019) implemented in the Lattigo library (Lattigo, 2022). This variant allows efficient polynomial multiplication by representing polynomials in both the RNS and NTT (Number Theoretic Transform) domains. We refer to Appendix A for more details of the full-RNS CKKS scheme.

### 3.5. Homomorphic matrix operation

Our optimization for the homomorphic matrix multiplication is derived from the scheme proposed by Jiang et al. (2018) for the CKKS scheme. They utilize a homomorphic linear transformation technique to build up the homomorphic matrix multiplication. We will introduce both of these techniques in the following.

#### 3.5.1. Homomorphic linear transformation

Halevi et al. first propose an approach to achieve linear transformations in the context of homomorphic encryption (Halevi and Shoup, 2014). They pointed out that any linear transformation $U\mathbf{m}$ can be represented as:

$$U \cdot \mathbf{m} = \sum_{0 \le \ell < n} (\mathbf{u}_\ell \odot \rho(\mathbf{m}; \ell)), \tag{1}$$

where $\mathbf{u}_\ell$ represents the $\ell$-th diagonal vector of $U$: $\mathbf{u}_\ell = (U_{0,\ell}, U_{1,\ell+1}, ..., U_{n-1,(\ell+n-1) \mod n})$. By associating $\mathbf{m}$ with the ciphertext and $U$ with the plaintext matrix in (1), we can effectuate a homomorphic linear transformation $U\mathbf{m}$ through $n$ ciphertext rotations, $n$ plaintext multiplication, and $n-1$ ciphertext additions. An algorithm known as Baby Step Giant Step (BSGS) can be employed to minimize the number of rotation operations involved (Halevi and Shoup, 2018): if there exists an arithmetic progression $\{s_k = a \cdot k | -d < k < d, d = d_1 d_2\}$ such that all indices $\ell$ corresponding to non-zero diagonal vectors in $U$ belong to this arithmetic progression, then Equation (1) can be reformulated as follows:

$$\begin{aligned} U\mathbf{m} &= \sum_{-d_2 < i < d_2, 0 \le j < d_1} \left( \mathbf{u}_{a\cdot(d_1 \cdot i + j)} \odot \rho(\mathbf{m}; a \cdot (d_1 \cdot i + j)) \right) \\ &= \sum_{-d_2 < i < d_2} \rho\left( \sum_{0 \le j < d_1} \left( \rho(\mathbf{u}_{a\cdot(d_1\cdot i+j)}; -a \cdot d_1 \cdot i) \odot \rho(\mathbf{m}; a \cdot j) \right); a \cdot d_1 \cdot i \right). \end{aligned} \tag{2}$$

Further optimization can be achieved by analyzing the rotation operations. Halevi et al. introduce a ciphertext rotation optimization technique called *hoisting* which reconstructs the internal operations for rotating a ciphertext $c = (c_0, c_1) \in R^2_{Q_l}$ by $k$ steps in the following sequence. (i) **Decompose**: Decompose $c_1$ into a vector $\mathbf{c}$ based on the decomposition base $\mathbf{b}$ of the rotation key $rtk_k$. (ii)**Permute**: Perform automorphism $\phi_k : X \to X^{5^k} (\mod X^N + 1)$ on each component of $\mathbf{c}$. (iii)**MultSum**: Perform the inner product $(c'_0, c'_1) \leftarrow \langle \mathbf{c}, \text{rtk}_k \rangle$, resulting in a *hoisted* ciphertext in $R^2_{Q_l P}$. (iv) **ModDown**: Reduce the modulus of the hoisted ciphertext to bring it back to $R^2_{Q_l}$. Here, **Decompose** and **ModDown** involve NTT and CRT basis extensions, which dominate the computational complexity of the rotation operation. The rotation key is only involved in the **MultSum**. Therefore, multiple rotations performed on the same ciphertext can share the result of **Decompose**(ct) when all the corresponding rotation keys are available.

Bossuat et al. apply the hoisting idea to the BSGS-optimized linear transformation algorithm (Bossuat et al., 2021). They apply the hoisting technique to the rotations $\rho(\mathbf{m}; a \cdot j), 0 \le j \le d_1$ in Equation (2) but do not perform the final step of the rotation, **ModDown**, immediately. Instead, they multiply the intermediate hoisted ciphertexts with the corresponding pre-rotated diagonal vectors $\rho(\mathbf{u}_{a\cdot(d_1\cdot i+j)}; -a \cdot d_1 \cdot i)$ and aggregate the products in $R^2_{PQ_\ell}$. **ModDown** is finally applied to the aggregated result. This optimization method is referred to as the *double-hoisting* technique. It reduces the computational complexity of the BSGS-optimized linear transformation from $(d_2 + d_1) \cdot$ (**MultSum** + **ModDown** + **Decompose** + **Permute**) to $(d_2 + d_1) \cdot$ (**MultSum** + **Permute**) + $(d_2 + 1) \cdot$ (**Decompose** + **ModDown**).

In the subsequent discussion, we assume that all the rotation keys required for performing any homomorphic encryption linear transformation are loaded into memory prior to the computation. This is the current implementation strategy in the Lattigo library. Although it is theoretically possible to load a key into memory only when it is needed for computation and release it soon afterward to control a smaller memory footprint for keys, we do not recommend this practice. Because it may incur additional time for disk read/write operations and disrupt the compactness of the storage structure. Especially, it will significantly slow down the speed of consecutive linear transformation computation.

#### 3.5.2. Homomorphic matrix multiplication by Jiang et al.

Jiang et al. proposed a homomorphic matrix multiplication approach based on the aforementioned linear transformation algorithm and its BSGS optimization (Jiang et al., 2018). A square matrix $A$ of size $n \times n$ can be encoded as a vector $\mathbf{a}$ with $n^2$ components using the row ordering encoding. Considering the multiplication of matrices $A$

and $B$, Jiang et al. first encode $A, B$ into vectors $\mathbf{a}$ $\mathbf{b}$ with row ordering, then compute the following equation:

$$\mathbf{c} = \sum_{k=0}^{n-1} (C^k Z\mathbf{a}) \odot (R^k T\mathbf{b}), \tag{3}$$

where $C$, $Z$, $R$, and $T$ are specific permutation matrices, and the $\mathbf{c}$ is the encoding vector of the matrix $AB$. They also design another permutation matrix transformation $G$ for matrix transposition: $\mathbf{a}' = G\mathbf{a}$, such that $\mathbf{a}'$ represents the transpose $A^T$ of the matrix $A$ represented by $\mathbf{a}$. The BSGS algorithm is employed for the transformations $Z, T$ and $G$ to greatly enhance efficiency. However, Jiang et al. treat BSGS as a black-box operation without considering internal optimization by hoisting techniques.

### 3.6. Principal component analysis using PowerMethod

Principal Component Analysis (PCA) is frequently employed for the dimensionality reduction of a dataset. Its fundamental concept involves identifying orthogonal axes that maximize the variance of the dataset projected onto these axes, which are referred to as principal components. Finding the top $k$ principal components is equivalent to determining the $k$ largest eigenvectors of the covariance matrix of the dataset.

The covariance matrix $\Sigma$ of the dataset $X$ is given by $\Sigma = \frac{1}{N} X^T X - \mu\mu^T$, where $u^T = \frac{1}{N}\sum_{i=0}^{N-1} \mathbf{x}_i^T$. The PowerMethod is an approximate algorithm for computing the dominant eigenvector of the covariance matrix (see Algorithm 1). It accepts a covariance matrix and its initial approximation of the dominant eigenvector and continuously applies the covariance matrix transformation to the approximate eigenvector to make it more and more accurate. The overflow should be prevented by normalizing the approximate vector after each matrix transformation. The EigenShift algorithm is used to shift the covariance matrix in terms of eigenvectors (see Algorithm 2). It takes the covariance matrix and the top $k$ eigenvectors as inputs and outputs the $k$-shifted covariance matrix, where the $(k+1)$-th dominant eigenvector of the original covariance matrix becomes the 1-th dominant eigenvector of the shifted one. By combining the PowerMethod and the EigenShift, we can compute the top principal components of the dataset.

**Algorithm 1** PowerMethod.

**Input:** $\Sigma$: covariance matrix of the dataset; $l_P$: number of iterations.
**Output:** $\mathbf{u}_1, \lambda_1$: dominant eigen-vector of $\Sigma$ and its eigen-value
1: Choose a random vector $\mathbf{v}^{(0)}$ of size $d$
2: **for** $i = 1$ to $l_P$ **do**
3: $\quad \mathbf{v}^{(i)} \leftarrow \Sigma \mathbf{v}^{(i-1)}$
4: $\quad \mathbf{v}^{(i)} \leftarrow \mathbf{v}^{(i)} / ||\mathbf{v}^{(i)}||$
5: **end for**
6: **return** $\mathbf{u}_1 = \mathbf{v}^{(l_P)}$ and $\lambda_1 = ||\mathbf{v}^{(l_P)}|| / ||\mathbf{v}^{(l_P-1)}||$

**Algorithm 2** EigenShift.

**Input:** $\Sigma$: covariance matrix of the dataset; $\{\mathbf{u}_i, \lambda_i\}$: the $i$-th dominant eigenvector of $\Sigma$ and its eigenvalue.
**Output:** $\Sigma_k$: $k$-shifted covariance matrix of $X$
1: **for** $i = 1$ to $k-1$ **do**
2: $\quad \Sigma_{i+1} = \Sigma_i - \lambda_i \mathbf{u}_i \mathbf{u}_i^T$
3: **end for**
4: **return** $\Sigma_k$

### 3.7. Iterative algorithm of inverse square root function

The inverse square-root (InvSRT) function is used to perform vector normalization in the PowerMethod algorithm. One feasible approach to evaluate InvSRT homomorphically is to use iterative algorithms such as Newton's Method (see Algorithm 3). These methods take the point $x$ for evaluation and an appropriate initial approximation of $\frac{1}{\sqrt{x}}$ as input and output a more accurate result after a finite number of iterations. We will use the term iterative InvSRT algorithm to refer to these methods in subsequent discussions. Providing a closer initial approximation for the iterative InvSRT algorithm can result in better convergence speed. Prior researches propose employing piecewise functions, Taylor expansions, and rational polynomials as functions to compute the initial approximation of the iterative InvSRT (Panda, 2022), (Qu and Xu, 2022).

**Algorithm 3** Newton's method.

**Input:** $x_0$: target evaluation point of inverse square-root function; $y_0$: initial approximation of $\frac{1}{\sqrt{x_0}}$
**Output:** $y_d$: more precise approximation of $\frac{1}{\sqrt{x_0}}$
1: **for** $i = 1$ to $d$ **do**
2: $\quad y_i \leftarrow \frac{1}{2} y_{i-1}(3 - x_0 y_{i-1}^2)$
3: **end for**
4: return $y_d$

## 4. Optimized homomorphic matrix multiplication

In this section, we improve the homomorphic matrix multiplication proposed by Jinag et al. to better serve our covariance matrix computation. Specifically, we first optimize the matrix multiplication with hoisting techniques to reduce its computation complexity (Section 4.1.1). Then, we analyze the space complexity of the hoisting-optimized matrix multiplication in parallel computation scenarios, during which we observe a problem that the large number of rotation keys required may occupy the memory and limit the performance of parallel computation (Section 4.1.2). To address this problem, we introduce techniques to reduce these rotation keys (Section 4.2). In the final subsection, we discuss the security of the proposed optimizations on homomorphic computations, affirming that they do not compromise the security of the underlying encryption scheme.

### 4.1. Homomorphic matrix multiplication equipped with hoisting techniques

#### 4.1.1. Applying hoisting techniques to linear transformation components

The hoisting techniques can be applied to the following linear transformations involved in the $n \times n$ matrix multiplication: $Z, T, \{C^k, R^k | 1 \le k < n\}$ (recall Equation (3)). We initially focus on $Z$ and $T$. $Z$ comprises $2n-1$ non-zero diagonal vectors with indices ranging from $-n+1$ to $n-1$. On the other hand, $T$ consists of only $n$ non-zero diagonal vectors, with indices $k \cdot n$ for $0 \le k < n$. Both $Z$ and $T$ exhibit a well-behaved arithmetic progression in the indices of their non-zero diagonal vectors. This property enables the BSGS algorithm to reduce the number of rotations from $O(n)$ to $O(n_1 + n_2)$, where $n_1, n_2$ denotes the inner and outer loop count. The double-hoisting technique can be naturally applied to the BSGS version of $Z$ and $T$, further reducing their rotation complexity.

The transformation $C^k$ has only two non-zero diagonal vectors: $k$ and $k-n$, while $R^k$ has only one non-zero diagonal vector: $n \cdot k$, for $1 \le k < n$. There is no need to apply BSGS optimization on these transformations due to the scarcity of their non-zero diagonals. Nonetheless, we can still enhance their efficiency using the hoisting technique. We observe that the rotation steps in the inner loop of $Z$ form a subsequence of the non-zero diagonal indices in the transformation set $\{C^k | 0 \le k < n\}$. Similarly, rotation steps used for the inner loop of $T$ form a subsequence of the non-zero diagonal indices in the set $\{R^k | 0 \le k < n\}$. This observation implies that the rotation keys of the double-hoisting $Z$ and $T$ are basically sufficient to perform the transformations $\{C^k, R^k | 0 \le k < n\}$ with the hoisting technique (column shifting transformations require an extra rotation step $-n$ to perform rotations of steps $k-n, k = 1, 2, \ldots, n-1$). In particular, we can achieve further optimization by emulating the strategy used in the double-hoisting technique for the transformations $C^k$. Deferring the final **ModDown** operations in the rotations associated with the two non-zero diagonal

**Table 1**
Rotation Complexity and Minimal Rotation Keys requirement Comparison between BSGS and dh-BSGS scheme for Z, T linTrans. $n_1$ and $n_2$ denote the inner loop count and the outer loop count in the BSGS algorithm. The complexity of the linear transformations is indicated by the ciphertext rotations performed. More specifically, these rotations are decomposed into their four internal operations: **MultSum**($MS$), **Permute**($Pm$), **Decompose**($Dp$), and **ModDown**($MD$).

| Scheme | LinTrans | MinRotKeys | Complexity (Rotations) |
|---|---|---|---|
| Origin | $Z$ | $2n_2+1$ | $(n_1+2n_2)\cdot(MS+Pm+Dp+MD)$ |
| Optimized | $Z$ | $n_1+2n_2-2$ | $(n_1+2n_2)\cdot(MS+Pm)+(2n_2+1)\cdot(Dp+MD)$ |
| Origin | $T$ | $n_2+1$ | $(n_1+n_2)\cdot(MS+Pm+Dp+MD)$ |
| Optimized | $T$ | $n_1+n_2-2$ | $(n_1+n_2)\cdot(MS+Pm)+(n_2+1)\cdot(Dp+MD)$ |
| Origin | $\{C^k \mid 0\le k<n\}$ | 2 | $(2n-1)\cdot(MS+Pm+Dp+MD)$ |
| Optimized | $\{C^k \mid 0\le k<n\}$ | $n_{Z1}+1$ | $(2n-1)\cdot(MS+Pm)+(2n/n_{Z1}+2)\cdot Dp+(2n/n_{Z1}+n)MD$ |
| Origin | $\{R^k \mid 0\le k<n\}$ | 1 | $(n-1)\cdot(MS+Pm+Dp+MD)$ |
| Optimized | $\{R^k \mid 0\le k<n\}$ | $n_{T1}$ | $(n-1)\cdot(MS+Pm)+(n/n_{T1}+1)\cdot Dp+(n-1)\cdot MD$ |

vectors, we multiply the two hoisted ciphertexts with the corresponding plaintext diagonal vectors in $R_{Q_l P}$ and aggregate the results. **ModDown** is then applied to the aggregated value.

We compared the ciphertext rotation complexity of various linear transformations in homomorphic matrix multiplication before and after applying the aforementioned hoisting technique (see Table 1). We can observe that the number of operations **Decompose** and **ModDown** is significantly decreased by the hoisting techniques. However, the minimal number of rotation keys required is increased by the inner loop counts of $Z$ and $T$. The increment of the rotation keys may cause memory limitation in a scenario where multiple matrix multiplication instances are desired to run in parallel. We will discuss this problem in the next subsection.

#### 4.1.2. Parallelizability and space complexity analysis

The matrix multiplication proposed by Jiang et al. exhibits parallelizability. Considering Equation (3), we can perform transformation sets $\{C^k Z\mathbf{a} \mid 0\le k<n\}$ and $\{R^k T\mathbf{b} \mid 0\le k<n\}$ in parallel with two threads. This idea can be extended to a *one-to-many* scenario where matrix $X$ is multiplied with multiple matrices $\{Y_i \mid i=1,2,\dots,d\}$. We can precompute and store $\{C^k Z\mathbf{x} \mid k=0,\dots,n\}$ (or $\{R^k T\mathbf{x} \mid k=0,\dots,n\}$), and perform the remaining steps of Equation (3) in parallel for each $Y_i$ with $d$ threads. This approach saves the time required for $d-1$ computations of $\{C^k Z\mathbf{x}\}$ (or $\{R^k T\mathbf{x}\}$). We can further consider multiple parallel one-to-many computations, which can save more time when $\{Y_i \mid i=1,2,\dots,d\}$ also need to be repeatedly involved in calculations with a set of $\{X_i \mid i=1,2,\dots,d\}$.

However, we concern that sufficient space for the aforementioned parallel computation of matrix multiplication cannot be easily obtained. The space complexity of matrix multiplication is determined by the rotation key space of $Z$ and $T$, as well as the intermediate ciphertexts cached by all parallel instances of transformations. On one hand, the total number of rotation keys required in matrix multiplication is $n_{Z1}+2n_{Z2}+n_{T1}+n_{T2}-3$. On the other hand, the number of intermediate ciphertexts that need to be cached increases with the number of parallel instances of matrix multiplication. For example, $d$ instances of matrix multiplication collectively require caching $n$ regular ciphertexts and $d\cdot n_{T1}$ (or $d\cdot n_{Z1}$) hoisted ciphertexts under the one-to-many scenario. The scalability of increasing parallel instances might be hindered by the large space requirement for rotation keys.

### 4.2. Rotation key reduction for homomorphic matrix multiplication

We aim to minimize the number of rotation keys for homomorphic matrix multiplication, thereby freeing up space for the parallel computation of more instances. We introduce two types of rotation key reduction techniques in the following subsection. The first technique is a simple key substitution method. For an arithmetic sequence of rotations applied on a ciphertext, the simple key substitution constructs this sequence of rotation by repeatedly performing rotation with one of its subsequences, resulting in a reduction in the number of required rotation keys. However, this approach compromises the benefits brought by the hoisting technique, leading to a decrease in the overall speed of matrix multiplication. The second technique involves applying a decomposition called diagonal convergence decomposition to the transformations $Z$ and $T$. The number of non-zero diagonal vectors in the decomposed transformations is significantly reduced, resulting in a notable reduction in the number of keys while preserving the integrity of hoisting.

#### 4.2.1. Simple key substitution method

Recall that we only utilize the rotation keys from the inner loop of transformation $T$ to perform all ciphertext rotations of the row transformations (see Section 4.1.1). This approach is an instance of the simple key substitution method which can be described as follows: when applying a sequence of rotations to a ciphertext, if the rotation steps form an arithmetic progression starting from 0, then any subsequence starting from 0 of that arithmetic progression can be used to perform the rotations. However, using shorter subsequence results in fewer benefits from the hoisting technique, as the hoisting technique cannot be fully deployed across the entire sequence but only within individual subsequences. Employing this method to reduce the rotation keys required for the inner loop of $Z$ (or $T$) from $n_1$ to $n_1'$ leads to an additional cost of $n_1/n_1'$ **Decompose** and **ModDown** operations and a subsequent effect on the row (or column) transformations. Moreover, the simple key substitution method has marginal effectiveness when the inner loop count of the linear transformation is relatively small. This is because fewer iterations in the inner loop imply an increase in the number of outer loop iterations and the corresponding required keys.

#### 4.2.2. Decomposition of diagonal vector convergence for specific permutation matrix

We present the second method for reducing the required rotation keys in matrix multiplication to overcome the limitations of the simple key substitution. Recall that the number of rotation keys required to perform a homomorphic linear transformation is essentially determined by the number of non-zero diagonal vectors in that linear transformation. This implies that if we can decompose $Z$ and $T$ into transformation with fewer non-zero diagonal vectors, we can reduce the number of rotation keys. Henceforth, we first establish a concept called diagonal convergence decomposition in Definition 1 to describe a decomposition method that reduces the number of non-zero diagonal vectors in linear transformations. This decomposition represents the original matrix as a sum and product of several matrices, such that the combined set of these decomposed matrices has fewer distinct non-zero diagonal indices compared to the original matrix. To better focus on the diagonals, we also define the diagonal coordinate of a matrix unit and the rules converting it back to the normal coordinate that locates units using the row and column index in Definitions 2, 3.

Constructing a diagonal convergence decomposition method that applies to all matrices might not be a straightforward task. Therefore, we will specifically focus on designing the diagonal convergence decomposition for $Z$ and $T$. Both $Z$ and $T$ are permutation matrices, which

are Boolean matrices with the properties that each row and each column contain exactly one 1 value. We observe that when decomposing a Boolean matrix $A$ of size $n \times n$ into a product of two Boolean matrices $A_2A_1$, for any non-zero diagonal $k$ in $A$ and its $i$-th element $A_{D(k,i)}$, if $A_{D(k,i)}$ is 1, there always exist a diagonal $k_1$ in $A_1$ and a diagonal $k_2$ in $A_2$ such that the elements in the $k_1$-th diagonal of $A_1$ are 1 in the same column as $A_{D(k,i)}$, the elements in the $k_2$-th diagonal of $A_2$ are 1 in the same row as $A_{D(k,i)}$, and $k = k_1 + k_2 \mod n$ (see Theorem 1). In Definition 4, we utilize Theorem 1 to construct a method for decomposing a single diagonal in $A$ into diagonals in $A_1$ and $A_2$.

**Definition 1.** We refer to a matrix decomposition as a **diagonal convergence decomposition** when it expresses a square matrix as a product and sum of a series of matrices, and the total number of distinct diagonal vector indices in these matrices is smaller than the number of distinct diagonal vector indices in the original matrix.

**Definition 2.** For any $n \times n$ square matrix, we represent its column indices, row indices, and diagonal vector indices using the additive group $(\mathbb{Z}/n\mathbb{Z}, +)$, where the indices of the diagonal vectors are defined based on the column indices of the elements in the first row of the matrix.

**Definition 3.** For an $n \times n$ square matrix, the element located at row $i$ and column $j$ possesses a *normal coordinate* $N(i,j) \in (\mathbb{Z}/n\mathbb{Z})^2$, and a *diagonal coordinate* $D(k,l) \in (\mathbb{Z}/n\mathbb{Z})^2$. Thus, there exists a mapping $f : (\mathbb{Z}/n\mathbb{Z})^2 \to (\mathbb{Z}/n\mathbb{Z})^2$ that transforms a normal coordinate to a diagonal coordinate:

$$\begin{aligned} f(N(i,j)) &= D(j-i,i), \\ f^{-1}(D(k,l)) &= N(l,k+l). \end{aligned} \tag{4}$$

We denote the symbol $A_{N(i,j)}$ to represent the element of $A$ located at the normal coordinate $N(i,j)$, and $A_{D(k,l)}$ to represent the element of $A$ located at the diagonal coordinate $D(k,l)$.

**Theorem 1.** *Let $A$ be a boolean matrix that can be decomposed into $A = A_2A_1$, where $A_1$ and $A_2$ are both boolean matrices. For any 1 element $A_{N(i,j)}$ in $A$, there exists a unique index $0 \le a < n$ in $A_1$ and $A_2$ such that $A_{1N(a,j)}$ and $A_{2N(i,a)}$ are both 1. Furthermore, considering $D(k,i) = f(N(i,j))$, $D(k_1,a) = f(N(a,j))$, and $D(k_2,i) = f(N(i,a))$, we have $k_1 + k_2 = k$.*

**Proof.** Given $A = A_2A_1$, we have $A_{N(i,j)} = \sum_{a=0}^{n-1} A_{1N(a,j)} \cdot A_{2N(i,a)}$. Since $A_1$ and $A_2$ are Boolean matrices, the equation $A_{N(i,j)} = 1$ holds only when there exists a unique pair $(A_{1N(a,j)}, A_{2N(i,a)})$ that equals $(1,1)$. According to Definition 3, we have $k_1 + k_2 = (j-a) + (a-i) = k$. □

**Definition 4.** For a boolean matrix $A$, $A_1$, and $A_2$ of size $n \times n$, decomposing the $k$th diagonal of $A$ onto the $k_1$th diagonal of the right matrix $A_1$, and the $k_2$th diagonal of the left matrix $A_2$ refers to the following process: For each 1 element $A_{D(k,l)}$ in the $k$th diagonal, set $A_{1D(k_1,k+l-k_1 \mod n)}$ to 1, which corresponds to the element on the $k_1$th diagonal with the same column index as $k$. Also, set $A_{2D(k_2,l)}$ to 1, which corresponds to the element on the $k_2$th diagonal with the same row index as $k$. Here, $k_1 + k_2 = k$.

Before we proceed with the diagonal convergence decomposition of matrices $Z$ and $T$, let us establish some common ground. Let us assume that the discussions henceforth are based on the $n \times n$ matrix multiplication scenario, meaning matrices $Z$ and $T$ are always of size $n^2 \times n^2$. Moreover, we define that if the maximum non-zero diagonal index of a matrix is $n' \le 0$, it is equivalent to stating that all non-zero diagonal vectors of the resulting matrices belong to the set $[-n', n'] \cap \mathbb{Z}$.

*Decomposition of linear transformation $Z$* The value of the $\ell$-th component in the $k$-th non-zero diagonal vector $\mathbf{z}_k[\ell]$ can be expressed as (all operations below involving non-zero diagonal vector indices are assumed to be in the residual class $(-n^2/2, n^2/2]$ of $(\mathbb{Z}/n^2\mathbb{Z}, +)$):

$$\mathbf{z}_k[\ell] = \begin{cases} 1, & \text{if } k \ge 0 \text{ and } 0 \le \ell - n \cdot k < (n-k); \\ 1, & \text{if } k < 0 \text{ and } -k \le \ell - (n+k) \cdot n < n; \\ 0, & \text{otherwise.} \end{cases} \tag{5}$$

We observe that the 1 values of $Z$ are symmetrically distributed on both sides of the 0th diagonal vector. They can be divided into $n$ submatrices of size $n \times n$, each having the 0th diagonal vector of $Z$ as its own 0th diagonal vector. These submatrices do not overlap with each other. Each submatrix contains $n$ elements with a value of 1. If we denote $S_i$, $0 \le i < n$, as the $n$ submatrices of diagonals of $Z$, then the 1 values in $S_i$ always fill the diagonals $i$ and $i-n$. Considering these characteristic, we design the diagonal convergence decomposition of $Z$ as follows:

1. Set the expected maximum non-zero diagonal index as $n'$. Divide $Z$ into the format $Z = Z_1 + Z_2$, where $Z_1$ is the matrix containing all non-zero diagonal vectors of $Z$ with an index no smaller than 0 and $Z_2$ contains the rest of the non-zero diagonal vectors. This step ensures that each submatrix of $Z_1$ and $Z_2$ has at most one non-zero diagonal vector, and each unit with value 1 belongs to a submatrix.
2. For both $Z_1$ and $Z_2$, perform the following decomposition (assuming $Z_1$ or $Z_2$ is denoted as $Z'$):
   (a) For each non-zero diagonal with index $k$ in $Z'$, If the absolute value of the index is greater than $n'$, decompose it onto the $k - \text{sign}(k) \cdot n'$ diagonal of the left matrix $Z'_l$ and the $\text{sign}(k) \cdot n'$ diagonal of the right matrix $Z'_r$. If the absolute value of the index is no bigger than $n'$, decompose it onto the $n'$ diagonal of the $Z'_l$ and the 0 diagonal of the $Z'_r$.
   (b) After iterating over all non-zero diagonals, if $Z'_l$ still contains non-zero diagonals greater than $n'$, replace $Z'$ with $Z'_l$ and repeat the decomposition. If no such diagonals exist, return the product of $Z'_l$ and all the right matrices obtained from the decomposition as the final decomposition result of $Z'$.

   Finally, we are able to decompose both $Z_1$ and $Z_2$ into the product of $\lceil \frac{n-1}{n'} \rceil$ matrix, respectively, as follows:

$$Z = \prod_{i=1}^{\lceil \frac{n-1}{n'} \rceil} Z_{1i} + \prod_{i=1}^{\lceil \frac{n-1}{n'} \rceil} Z_{2i}. \tag{6}$$

The decomposed expressions of both products have the same characteristic: only the leftmost matrix has non-zero diagonal vectors with indices in the interval $[-n'+1, n'-1] \cap \mathbb{Z}$, while the remaining matrices have only non-zero diagonal vectors with indices $-n', 0$ (or $n', 0$). Therefore, the dominant complexity lies in the two leftmost linear transformations of the two matrix products. These two linear transformations still possess the same arithmetic progression property as $Z$, thus making them suitable for benefiting from the double-hoisting BSGS algorithm.

*Decomposition of linear transformation $T$* Transformation $T$ consists of $n$ non-zero diagonal vectors, whose indices are $\{k \cdot n | 0 \le k < n\}$. For the $k \cdot n$-th non-zero diagonal vector $\mathbf{t}_{kn}$ of $T$, the value of its $\ell$-th component $\mathbf{t}_{kn}[\ell]$ can be expressed as (all operations below involving diagonal vector indices are assumed to be in the residual class $[0, n^2-1]$ of $(\mathbb{Z}/n^2\mathbb{Z}, +)$):

$$\mathbf{t}_{kn}[\ell] = \begin{cases} 1, & \text{if } \ell \in \{k + n \cdot i | 0 \le i < n\}\ ; \\ 0, & \text{otherwise.} \end{cases} \tag{7}$$

Although the units with a value of 1 in $T$ can not be divided into submatrices like $Z$, the non-zero diagonals of $T$ have a consistent spacing between the units with a value of 1. Specifically, we observe that the row indices of units with a value of 1 in the same non-zero diagonal

**Table 2**
Rotation Complexity and Minimal Rtks Number Comparison of LinTrans $Z, T$ among original double-hoisting BSGS(dh-BSGS), dh-BSGS with Simple Key Substitution (SmpKeySub) and dh-BSGS with Diagonal Vector Convergence Decomposition (DCDmp). **MaxDiagNo.** denotes the maximum non-zero diagonal index in the lineartransformation(s), and **MinRotKeys** indicates the minimum number of keys required to perform the lineartransformation(s).

| Scheme | LT | MaxDiagNo. | MinRotKeys | Complexity (Rotations) |
|---|---|---|---|---|
| dh-BSGS | $Z$ | $n-1$ | $n_1+2n_2-2$ | $(n_1+2n_2)\cdot(MS+Pm)+(2n_2+1)\cdot(Dp+MD)$ |
| SmpKeySub | $Z$ | $n-1$ | $n_1'+2n_2-2$ | $(n_1+2n_2)\cdot(MS+Pm)+(2n_2+n_1/n_1'+1)\cdot(Dp+MD)$ |
| DCDmp | $Z$ | $n'=n_1\cdot n_2'$ | $n_1+2n_2'-1$ | $2(n_1+n_2'+n/n'-1)\cdot(MS+Pm)+2(n_2'+n/n')\cdot(Dp+MD)$ |
| dh-BSGS | $T$ | $(n-1)\cdot n$ | $n_1+n_2-2$ | $(n_1+n_2)\cdot(MS+Pm)+(n_2+1)\cdot(Dp+MD)$ |
| SmpKeySub | $T$ | $(n-1)\cdot n$ | $n_1'+n_2-2$ | $(n_1+n_2)\cdot(MS+Pm)+(n_2+n_1/n_1'+1)\cdot(Dp+MD)$ |
| DCDmp | $T$ | $n'\cdot n=n_1\cdot n_2'\cdot n$ | $n_1+n_2'-1$ | $(n_1+n_2'+n/n'-1)\cdot(MS+Pm)+(n_2'+n/n')\cdot(Dp+MD)$ |

**Algorithm 4** Diagonal Convergence Decompose for LinTrans Z (DCDforZ).

**Input:**
$n$: dimension parameter to create a $Z$ with size $n^2\times n^2$;
$n'$: the target maximum non-zero diagonal index.

**Output:** $\{Z_{1i}, Z_{2i} | Z=\prod_{i=1}^{\lceil\frac{n-1}{n'}\rceil} Z_i\}$: decomposed matrices.

1: Assume all indices of diagonal vectors are in residual class $(-n^2/2, n^2/2]$.
2: Set $Z_1$ as the matrix containing all non-zero diagonal vectors of $Z$ with an index no smaller than 0, and set $Z_2$ as the matrix containing the rest of the non-zero diagonal vectors of $Z$.
3: **for** t = 1,2 **do**
4:  **for** $i=1$ to $\lceil\frac{n-1}{n'}\rceil-1$ **do**
5:  Initialize matrices $Z_t^{(l)}, Z_t^{(r)}$ as zero matrices.
6:  **for** each non-zero diagonal vector with index $|k|\geq n'$ in $Z_t$ **do**
7:  Decompose the vector to the $n'$-th non-zero diagonal vector of the right matrix $Z_t^{(r)}$ and the $(k-\text{sign}(k)\cdot n')$-th diagonal of the left matrix $Z_t^{(l)}$.
8:  **end for**
9:  **for** each non-zero diagonal vector $k<n'$ in $Z_t$ **do**
10:  Decompose the vector to the 0-th non-zero diagonal vector of the right matrix $Z_t^{(r)}$ and the $k$-th diagonal of the left matrix $Z_t^{(l)}$.
11:  **end for**
12:  (At this point, $Z_t=Z_t^{(l)}Z_t^{(r)}$)
13:  **if** $i\neq\lceil\frac{n-1}{n'}\rceil-1$ **then**
14:  $Z_{t(\lceil\frac{n-1}{n'}\rceil-i+1)}\leftarrow Z_t^{(r)}$ , $Z_t\leftarrow Z_t^{(l)}$
15:  **else**
16:  $Z_{t(\lceil\frac{n-1}{n'}\rceil-i+1)}\leftarrow Z_t^{(r)}$ , $Z_{t1}\leftarrow Z_t^{(l)}$
17:  **end if**
18:  **end for**
19: **end for**
20: **return** $\{Z_{1i}, Z_{2i} | Z=\prod_{i=1}^{\lceil\frac{n-1}{n'}\rceil} Z_i\}$

vector are congruent modulo $n$. This implies that the row sequence of 1-valued units on the $kn$-th non-zero diagonal of $T$ forms a coset $k+\langle n\rangle$ of an $n$-order subgroup $\langle n\rangle$ in $\mathbb{Z}/n^2\mathbb{Z}$. According to Definition 3, their column sequences also form the same coset. If we attempt to decompose any diagonal $kn$ in $T$ onto a diagonal of index divisible by $n$ in the right matrix $T_1$ and the left matrix $T_2$, then the 1-valued units on these two diagonals in the left and right matrices will inherit the property of forming the coset $k+\langle n\rangle$. Furthermore, if any two diagonals $kn$ and $k'n$ with the coset properties $i+\langle n\rangle$ and $j+\langle n\rangle$, respectively, are decomposed onto the $k_1n$ diagonal of the right matrix (or left matrix), then the 1-valued units in that diagonal will form a set $(i+\langle n\rangle)\cup j+\langle n\rangle)$. Based on these observations, we can repeatedly decompose the diagonals in $T$ that have indices greater than the expected maximum non-zero diagonal index $n\cdot n'$ into diagonals $k-n\cdot n'$ and $n\cdot n'$, ultimately decomposing $T$ into the product of $\lceil\frac{n-1}{n'}\rceil$ matrices:

$$T=\prod_{i=1}^{\lceil\frac{n-1}{n'}\rceil} T_i. \tag{8}$$

Similar to the decomposition of Z, only the leftmost matrix in the product contains non-zero diagonals with indices in $\{kn | k=0,1,\ldots,n'\}$, while all other right matrices have only non-zero diagonals with indices $n'n$ and 0. This implies that the complexity of the product is dominated by the complexity of the leftmost linear transformation, whose diagonal indices belong to a smaller range compared to the original approach. The detailed decomposition process is provided in Algorithm 5.

**Algorithm 5** Diagonal Convergence Decompose for LinTrans T (DCDforT).

**Input:**
$n$: the dimension parameter for creating a $T$ with size $n^2\times n^2$;
$n'\cdot n$: the target maximum non-zero diagonal index.

**Output:** $\{T_l | T=\prod_{i=1}^{\lceil\frac{n-1}{n'}\rceil} T_i\}$ decomposed matrices of linear transformation $T$.

1: Assume that all indices of diagonal vectors are in residual class $[0, n^2-1]$
2: **for** $i=1$ to $\lceil\frac{n-1}{n'}\rceil-1$ **do**
3:  Initialize $T^{(0)}$ and $T^{(1)}$ as zero matrices.
4:  Traverse the $i\cdot n$-th non-zero diagonal of $T$, where $0\leq i\leq n'$, and decompose it onto the 0th diagonal of the right matrix $T^{(r)}$ and the $i\cdot n$-th diagonal of the left matrix $T^{(l)}$.
5:  Traverse the $i\cdot n$-th non-zero diagonal of $T$, where $n'<i\leq(n-1)$, and decompose it onto the $n'\cdot n$-th diagonal of the right matrix $T^{(r)}$ and the $(i-n')\cdot n$-th diagonal of the left matrix $T^{(l)}$.
6:  (At this point, $T=T^{(l)}T^{(r)}$)
7:  **if** $i\neq\lceil\frac{n-1}{n'}\rceil-1$ **then**
8:  $T_{\lceil\frac{n-1}{n'}\rceil-i+1}\leftarrow T^{(r)}$, $T\leftarrow T^{(l)}$
9:  **else**
10:  $T_{\lceil\frac{n-1}{n'}\rceil-i+1}\leftarrow T^{(r)}$, $T_1\leftarrow T^{(l)}$
11:  **end if**
12: **end for**
13: return $\{T_l | T=\prod_{i=1}^{\lceil\frac{n-1}{n'}\rceil} T_i\}$

After formulating the diagonal convergence decomposition for $Z$ and $T$, we proceed to analyze its impact on the number of rotation keys and rotation complexity. Let $n_1$ and $n_2$ continue to represent the inner loop and outer loop counts of the original double-hoisting BSGS applied to $Z$ (or $T$). Since both the diagonal convergence decompositions for $Z$ and $T$ are able to reduce the non-zero diagonal vectors from $n-1$ to $n'$, for a fixed $n_1$, the decomposition shrinks the number of keys required for the outer loop (of $Z$ and $T$) with a factor of $\frac{n_2'}{n_2-1}$, where $n_2'=n'/n_1$. As for the rotation complexity, the primary complexity for $Z$ and $T$ after decomposition arises from the leftmost matrices in the matrix products, which require $(n_1+n_2')(MS+Pm)+(n_2'+1)(Dp+MD)$. The extra $(n/n'-1)\cdot(MS+Pm+Dp+MD)$ is attributed to the $n/n'-1$ matrices on the right side. We compare the number of rotation keys and computational complexity (in terms of internal rotation operations) among the diagonal convergence decomposition, simple key substitution method, and the dh-BSGS algorithm without any key simplification strategy for the $Z$ and $T$ transformations, as shown in Table 2. More detailed experimental data obtained from specific configurations will be discussed in the implementation section. Additionally, we present the complete process of ciphertext decomposition matrix multiplication with diagonal convergence decomposition in Algorithm 6, omitting internal optimization details such as double-hoisting and hoisting, and providing a high-level illustrative flow.

### 4.3. Security discussion

**Theorem 2.** *The homomorphic encryption scheme, enhanced with the homomorphic matrix multiplication with proposed hoisting optimization and*

**Algorithm 6** Matrix Multiplication using Diagonal Convergence Decomposition (DMatrixMult).

**Input:**
ct$(A)$, ct$(B)$: ciphertexts encrypting matrices $A$ and $B$ with size $n \times n$;
$n'_Z$: expected maximum non-zero diagonal index for $Z$;
$n \cdot n'_T$: expected maximum non-zero diagonal index for $T$.
**Output:** ;
ct$(AB)$: encrypted multiplication result between matrices $A$ and $B$
$\{Z_{1i}\}, \{Z_{2i}\} \leftarrow$ DCDforZ$(n^2, n'_Z)$
$\{T_i\} \leftarrow$ DCDforT$(n^2, n \cdot n'_T)$
$ct(A_1), ct(A_2) \leftarrow ct(A)$
$ct(B^{(0)}) \leftarrow ct(B)$
**for** $Z_{1i}$ in $\{Z_{1i}\}$ **do**
  ct$(A_1) \leftarrow$ LinTrans(ct$(A_1), Z_{1i}$)
**end for**
**for** $Z_{2i}$ in $\{Z_{2i}\}$ **do**
  $ct(A_2) \leftarrow$ LinTrans$(ct(A_2), Z_{2i})$
**end for**
$ct(A^{(0)}) \leftarrow$ Add$(ct(A_1), ct(A_2))$
**for** $T_i$ in $\{T_i\}$ **do**
  ct$(B^{(0)}) \leftarrow$ LinTrans(ct$(B^{(0)}), T_i$)
**end for**
ct$(AB) \leftarrow$ Mul(ct$(A^{(0)})$, ct$(B^{(0)})$)
**for** $1 \le k < n$ **do**
  ct$(A^{(k)}) \leftarrow$ LinTrans(ct$(A^{(0)}, C^k)$
  ct$(B^{(k)}) \leftarrow$ LinTrans(ct$(B^{(0)}, R^k)$
  ct$(AB^{(k)}) \leftarrow$ Mul(ct$(A^{(k)})$, ct$(B^{(k)})$)
  ct$(AB) \leftarrow$ Add(ct$(AB^{(k)})$, ct$(AB)$)
**end for**
return ct$(AB)$

*rotation key reduction techniques, maintains an equivalent level of security to the original scheme.*

**Proof.** Our aforementioned modifications on the homomorphic multiplication exclusively manipulate existing ciphertexts and do not involve any participation of private keys. This implies that our revisions are independent of the security assumptions, encryption, and decryption processes of the underlying homomorphic encryption scheme. □

According to Theorem 2, the IND-CPA nature of the homomorphic encryption scheme will not be affected by our modified operations as long as its ciphertexts are originally pseudorandom. Theoretically, any entity that has access to corresponding public keys may perform these operations on ciphertexts, but the entity's knowledge of the plaintext contained in the ciphertext remains unchanged before and after the operations.

## 5. Homomorphic covariance matrix computation for large datasets

In this section, We move on to present the method to pack large datasets into ciphertexts which enables seamless execution of the homomorphic matrix multiplication (see Section 5.1) and a homomorphic circuit that efficiently computes the covariance matrix of the packed and encrypted dataset, see Section 5.2.

### 5.1. Ciphertext packing

For a given power of two, $N$, CKKS allows the encryption of a vector in $\mathbb{C}^{N/2}$, as a whole to obtain a ciphertext (please refer to Appendix A for more detail). Operations on the ciphertext may induce corresponding computations on the underlying plaintext. Therefore, how we partition and package our dataset into ciphertexts significantly impacts the efficiency of privacy-preserving computations. Let $X$ be an $s \times t$ dataset, where $s$ represents the number of samples and $t$ represents the number of features. We partition $X$ into square submatrices with size $n \times n$, where $n^2$ equals the CKKS scheme parameter $N/2$. This allows us to encode each submatrix into one ciphertext. Features of the dataset are divided into $k$ partitions, where $k = \lceil \frac{t}{n} \rceil$. In this way, each ciphertext will contain one partition of features from $n$ samples/records. Note that $t$ is not always divisible by $n$, so zero-padding is necessary. Two possible methods for padding are as follows:

1. The first $k-1$ columns of submatrices each contain $n$ features, while the last column of submatrices contains the last $t - k \cdot n$ features and pads the rest of its space with zeros. This approach expands the feature count from $t$ to $k \cdot n$ and treats each ciphertext as an $n \times n$ matrix representing $n$ samples with $n$ features.
2. All columns of submatrices store only $n - p$ valid features and pad the rest of their space with zeros, where $p = \lfloor (t - k \cdot n)/k \rfloor$. This approach allows each ciphertext to be viewed as an $n \times (n-p)$ matrix, taking advantage of some possible optimizations for rectangular matrix multiplication while still enabling operations on square matrices.

In summary, $\lceil s/n \rceil \cdot k$ ciphertexts are required to pack the entire dataset $X$.

### 5.2. Homomorphic covariance matrix computation for large datasets

We present a method for computing the covariance matrix of an encrypted dataset. Assume that we have partitioned $X$ into $\lceil s/n \rceil \cdot k$ ciphertexts, with each ciphertext representing an $n \times n$ submatrix of $X$. We denote $X_i$ as the $i$-th column submatrix of $X$, where $0 \le i < k$. We also denote $X_i[\ell]$ as the $\ell$-th submatrix of $X_i$ and ct$(X_i[\ell])$ as the ciphertext of $X_i[\ell]$. We need to calculate the mean 'vector' $\mu$ of the dataset if it is not centered. $\mu$ is represented by $k$ submatrices, where each column contains $n$ replication of one feature's mean value. $0 \le i < k$, denote $\mu[i]$ as the $i$-th submatrix of the mean vector. $\mu$ can be computed through the following equation:

$$\mu[i] = \text{Aggregate}\left( \sum_{0 \le \ell < \lceil s/n \rceil} X_i[\ell]; 0 \right), \tag{9}$$

where Aggregate$(X; \text{axis}), \text{axis} \in 1, 0$ represents aggregating a matrix $X$ by columns (axis $= 1$) or by rows (axis $= 0$). This algorithm requires at most 1 multiplication depth (see Algorithm 7).

**Algorithm 7** Aggregate.

**Input:**
$ct(X)$: encrypted matrix $X$ with size row $\times$ col;
axis: aggregation axis.
**Output:**
$ct($Aggregate$(X; \text{axis}))$
$ct(A) \leftarrow ct(X)$
**if** axis == 0 **then**
  **for** $i \leftarrow col; i <$ row $\cdot$ col$; i = (i << 1)$ **do**
    $ct(A) \leftarrow$ Add(Rot$(ct(A); i), ct(A))$
  **end for**
**end if**
**if** axis == 1 **then**
  **for** $i \leftarrow 1; i <$ col$; i = (i << 1)$ **do**
    $ct(A) \leftarrow$ Add(Rot$(ct(A); i), ct(A))$
  **end for**
  $ct(A) \leftarrow$ Mult$(ct(A), pt(M))$, where $M$ is a mask matrix with the same dimensions as $X$, except for the first column elements which are all 1 and other elements are 0
  **for** $i := -1; i > -col; i = (i << 1)$ **do**
    $ct(A) \leftarrow$ Add(Rot$(ct(A); i), ct(A))$
  **end for**
**end if**
**return** ct$(A)$

Next, matrix $\mu^T \mu$ can be simply computed by transpositions and coordinate-wise multiplications:

$$\mu^T \mu_i[j] = \mu[j]^T \odot \mu[i]. \tag{10}$$

We then move on to compute each submatrix unit in $X^T X$. For the submatrix $X^T X_i[j]$, we iterate through each submatrix in column $X_i$

and multiply it with the transposition of the corresponding submatrix in column $X_j$:

$$X^T X_i[j] = \sum_{0 \le \ell < \lceil s/n \rceil} X_j[\ell]^T X_i[\ell]. \tag{11}$$

Here, each matrix multiplication can be performed in ciphertext space using our improved homomorphic matrix multiplication. If $X$ is already centered, then $X^T X$ represents the covariance matrix. Otherwise, $X^T X$ should be subtracted by $\mu^T \mu$ to become the covariance matrix. It is worth noting that directly computing the covariance matrix homomorphically using the equation above is not the most efficient implementation. We provide the following optimizations:

1. Notice that for a given $0 \le j < k$, there is a one-to-many matrix multiplication scenario between $X_j[\ell]$ and matrix set $\{X_i[\ell] | 0 \le i < k\}$ in the equation above. We can perform the inner product operations of $k$ columns of submatrices in parallel: $\sum_{0 \le \ell < \lceil s/n \rceil} X_j[\ell]^T X_i[\ell], 0 \le i < k$, and save time by storing $\{C^k Z \mathrm{ct}(X_j[\ell]^T) | 0 \le \ell < \lceil s/n \rceil\}$.
2. We can further cache all $\{R^k T ct(X_i[\ell]) | 0 \le \ell < \lceil s/n \rceil, 0 \le j < k\}$, so that each $X_i[\ell]$ only needs to undergo one $T$ transformation and $n-1$ row transformations throughout the entire covariance matrix computation process.
3. Based on the fact that the covariance matrix is a symmetric matrix, it suffices to calculate the results of one side of the diagonal and transpose them to obtain the results for the other side, saving nearly half of the covariance computation time.

However, the ciphertexts required to be stored in Optimization 1 and 2 usually cannot be fully cached in memory due to space limitations. Therefore, it is often necessary to split the dataset into suitable segments by rows and apply Optimization 1 and 2 segment by segment.

We present the computation of the entire encrypted covariance matrix with the aforementioned Optimization 3 in Algorithm 8. Optimization 1 and 2 are not explicitly shown in the algorithm since they involve restructuring the internal computation order of the DMtrixMult function. Theorem 3 provides the multiplication depth and computation complexity of Algorithm 8, where the computation complexity is measured at the granularity of matrix multiplications since matrix multiplication can be implemented using different methods with varying complexities (such as whether to use Optimization 1 and 2).

**Theorem 3.** *The multiplication depth of Algorithm 8 is* $2 + max(\lceil \frac{(n-1)}{n_Z} \rceil, \lceil \frac{(n-1)*n}{n_T} \rceil) + 2$, *and the time complexity is dominated by the number of matrix multiplications, which is* $O(\lceil \frac{k^2}{2} \rceil \cdot \lceil s/n \rceil)$.

**Proof.** We need 2 multiplication depths for matrix transpositions, 2 depths for column shiftings and ciphertext multiplications in the DMatrixMult function, and $k = max(k_1, k_2)$ depths for the linear transformations $Z$ and $T$, where $k_1 = \lceil \frac{(n-1)}{n_Z} \rceil$ is the number of depth used for decomposed $Z$, and $k_2 = \lceil \frac{(n-1) \cdot n}{n_T} \rceil$ is the number of depth used for decomposed $T$. □

## 6. Privacy-preserving PCA using homomorphic PowerMethod

We present the homomorphic evaluation circuit of the PowerMethod algorithm in the first subsection and demonstrate the process of performing privacy-preserving PCA utilizing our homomorphic PowerMethod circuit in the second subsection. In the final subsection, we provide comprehensive security proof for our proposed privacy-preserving PCA.

**Algorithm 8** Homomorphic Covariance Matrix Computation (HCovMtrx).

**Input:**
$\{\mathrm{ct}(X_j[\ell]) | 0 \le j < k, 0 \le \ell < \lceil s/n \rceil\}$: partitioned and encrypted dataset;
$n_Z$: maximum diagonal index for transformation $Z$;
$n_T$: maximum diagonal index $n_T$ for transformation $T$;
**Output:**
$\{\mathrm{ct}(\mathrm{Covi}[j]) | 0 \le i, j < k\}$: partitioned and encrypted covariance matrix.

**for** $0 \le j < k$ **do**
  Take $\mathrm{ct}(X_j[\ell]), 0 \le \ell < \lceil s/n \rceil$, $\mathrm{ct}(\mu[j]) \leftarrow \sum_{0 \le \ell < \lceil s/n \rceil} X_i[\ell]$
  **for** $n \le t < n^2$ **do**
    $ct(\mu[j]) \leftarrow \mathrm{Add}(\mathrm{ct}(\mu[j]), \mathrm{Rot}(\mathrm{ct}(\mu[j]); t))$
    $t \leftarrow t << 1$
  **end for**
  $\mathrm{ct}(X_j[\ell]^T) \leftarrow \mathrm{LinTrans}(\mathrm{ct}(X_j[\ell]), \mathrm{G})$ {perform transposition with permutation matrix $G$ in parallel for $0 \le \ell < \lceil s/n \rceil$}
  **for** $0 \le i < j$ **do**
    $\mathrm{ct}(X_j[\ell]^T X_i[\ell]) \leftarrow \mathrm{DMtrxMul}(\mathrm{ct}(X_j[\ell]^T), \mathrm{ct}(X_i[\ell]), n, n_Z, n_T)$ {compute in parallel for $0 \le \ell < \lceil s/n \rceil$}
    $\mathrm{ct}(X^T X_i[j]) \leftarrow \sum_{0 \le j < k} \mathrm{ct}(X_j[\ell]^T X_i[\ell])$; {aggregate the parallel computation results}
  **end for**
**end for**
**for** $0 \le j < k$ **do**
  **for** $j \le i < k$ **do**
    $ct(X^T X_i[j]) \leftarrow \mathrm{LinTrans}(ct(X^T X_j[i]), \mathrm{G})$
  **end for**
**end for**
**for** $0 \le j < k$ **do**
  $\mathrm{ct}(\mu[j]^T) \leftarrow \mathrm{LinTrans}(\mathrm{ct}(\mu[j]), T)$
  **for** $0 \le i < k$ **do**
    $\mathrm{ct}(\mu^T \mu_i[j]) \leftarrow \mathrm{Mul}(\mathrm{ct}(\mu[j]^T), \mathrm{ct}(\mu[i]))$
    $\mathrm{ct}(\mathrm{Cov}_i[j]) \leftarrow \mathrm{Add}(ct(X^T X_i[j]), ct(-\mu^T \mu_i[j]))$
  **end for**
**end for**
return $\{\mathrm{ct}(\mathrm{Cov}_i[j]) | 0 \le i, j < k\}$

### 6.1. Homomorphic PowerMethod circuit

We divide the demonstration of the homomorphic PowerMethod into three pivotal components: (i) the continuous transformation of the covariance matrix, (ii) the normalization of the approximate eigenvector, and (iii) the computation of the eigenvalue corresponding to the eigenvector.

#### 6.1.1. Covariance matrix transformation

For a column vector (or row vector) with $t$ components ($t$ is denoted as the number of features in Section 5.1), we replicate it into a column (or row) of submatrices using the following approach:

1. Divide the vector evenly into $k$ segments where each segment contains $n$ entries. The zero padding strategy is needed and should be consistent with the one used in ciphertext packing when $t$ is not divisible by $k$.
2. Horizontally (or vertically) replicate these $k$ segments of column vectors (or row vectors) $n$ times to construct $k$ submatrices.

The continuous linear transformation will be alternately applied to the horizontally replicated vector and the vertically replicated vector. Let $V_{t-1}$ be a set of sub-matrices representing a replicated column vector $\mathbf{v}_{t-1}$. For $0 \le i < k$, the covariance matrix transformation $\mathbf{v}_t = \mathrm{Cov}\mathbf{v}_{t-1}$ can be computed as:

$$\begin{aligned} V_t[i] &= \mathrm{Aggregate}\left( \sum_{0 \le j < k} (\mathrm{Cov}_j[i] \odot V_{t-1}[j]^T)^T; 0 \right) \\ &= \mathrm{Aggregate}\left( \sum_{0 \le j < k} \mathrm{Cov}_j[i]^T \odot V_{t-1}[j]; 0 \right). \end{aligned} \tag{12}$$

Here, $V_t = \{V_t[i] | 0 \le i < k\}$ represent the replicated row vector $v_t^T$. If $V_{t-1}$ represents a replicated row vector $\mathbf{v}_{t-1}^T$, then for $0 \le i < k$, the: covariance matrix transformation $\mathbf{v}_t^T = \mathbf{v}_{t-1}^T \mathrm{Cov}$ can be computed as:

$$V_t[i] = \text{Aggregate}\left(\sum_{0\le j<k} V_{t-1}[j] \odot \text{Cov}_i[j]^T; 1\right), \tag{13}$$

where $V_t$ represents the replicated column vector $\mathbf{v}_t$. This equation holds because the covariance matrix Cov is symmetric, i.e., $\text{Cov}_j[i]^T = \text{Cov}_i[j]$. Now We can perform the covariance matrix transformation on a vector continuously using these two equations alternately.

The entire process of the continuous covariance matrix transformation can be described as the following. At the onset of the PowerMethod iteration, we initially select a row vector as the approximate eigenvector, denoted as $v_0$. We replicate it into a row of submatrices denoted as $V_0$. The set of submatrices representing the approximate eigenvector in the $t$-th iteration is computed as:

$$\begin{aligned} V_t[i] =& \text{Aggregate}\left(\sum_{0\le j<k} V_{t-1}[j] \odot \text{Cov}_j[i]; t+1 \bmod 2\right) \cdot (t+1 \bmod 2) + \\ & \text{Aggregate}\left(\sum_{0\le j<k} V_{t-1}[j] \odot \text{Cov}_i[j]; t \bmod 2\right) \cdot (t \bmod 2). \end{aligned} \tag{14}$$

#### 6.1.2. Vector normalization

Vector normalization should be performed in each iteration of the PowerMethod to the length of the eigenvectors from continually increasing or decreasing, which could lead to overflow. Normalizing a vector $\mathbf{v}$ is done by applying a scaling factor of $\frac{1}{||\mathbf{v}||_2}$ to each component of the vector. Specifically, the scaling factor is obtained by evaluating the InvSRT function at the point $\langle \mathbf{v}, \mathbf{v} \rangle$. We mentioned that we can only use the iterative InvSRT algorithm to approximate the effect of InvSRT in the homomorphic encryption scenario (see Section 3.7). We summarize the following factors that significantly influence the accuracy of the iterative InvSRT algorithm:

1. Initial approximation: Given a fixed number of iterations, a closer initial approximation to the exact value yields more accurate results from the iterative algorithm. Conversely, an initial estimate that deviates significantly from the exact value may prevent the iterative algorithm from converging. Since the function $\frac{1}{\sqrt{x}}$ exhibits significant changes in slope on both sides of 1, resembling an "L" shape, determining an initial approximation that simultaneously approximates the exact value on both sides of 1 using a simple function (such as a low-degree polynomial) may be insufficient. However, constructing an initial approximation evaluation method that closely approximates the exact value on both sides of the evaluation interval can itself be a complex task. For example, approaches such as the piecewise function used in Panda (2022) or the rational polynomial function discussed in Qu and Xu (2022) yield initial estimates that are closer to the exact value. However, even these functions require an iterative approximation in the homomorphic setting.
2. Number of iterations: The more iterations performed, the closer the output of the iterative InvSqrt algorithm will be to the exact value.

To set up a good iterative InvSRT algorithm for the vector normalization in the homomorphic PowerMethod, our first step is to determine the upper bound of the norm of the approximate eigenvectors in order to establish the evaluation interval for the iterative InvSRT. Subsequently, we propose a Lazy Normalization strategy for the approximate eigenvector normalization in the PowerMethod. This strategy tolerates imprecise outputs of the iterative InvSRT algorithm within the input interval (0,1] during some rounds of the PowerMethod. However, it aims to achieve highly accurate outputs in the final iteration to ensure that the resulting eigenvectors are normalized.

*Estimating the upper bound of the norm* After the dataset has undergone normalization and centering, we assume that all of its units are bounded within $b > 0$. For each element in the covariance matrix Cov, we have the bound $|\text{Cov}_{ij}| = \frac{1}{n}|\langle \mathbf{x}_i, \mathbf{x}_j \rangle| \le b^2$, where $\mathbf{x}_i, 0 \le i < d$ represents the vector composed of the $i$-th feature values of all samples in the dataset. Therefore, the Euclidean norm of vector $\text{Cov}_i$ is bounded by $||\text{Cov}_i||_2 \le \sqrt{b^4 \cdot d}$. With this information, we can estimate the upper bound of the norm for an approximate eigenvector $\mathbf{y}$ produced by the covariance matrix transformation $\mathbf{y} = \text{Cov} \cdot \mathbf{v}$ with the following inequality:

$$\begin{aligned} ||\mathbf{y}||_2 &= \sqrt{\sum_{i=1}^{d} |y_i|^2} \\ &= \sqrt{\sum_{i=1}^{d} (\text{Cov}_{i1} v_1 + \text{Cov}_{i2} v_2 + \cdots + \text{Cov}_{id} v_d)^2} \le b^2 \cdot c \cdot d, \end{aligned} \tag{15}$$

where the Euclidean norm of the input approximate eigenvector $\mathbf{v}$ is bounded by $c$. In practical applications, different feature columns $\mathbf{x}_i$ may have different bounds $b_i$. If the cloud service provider has legal access to these bounds, tighter upper bounds can be constructed. In this case, we have $\text{Cov}_{ij} = \frac{1}{n}|\langle \mathbf{x}_i, \mathbf{x}_j \rangle| \le b_i b_j$, $||\text{Cov}_i||2 \le \sqrt{\sum_{0\le j<d} b_i^2 b_j^2}$, and the upper bound of the Euclidean norm of $\mathbf{y}$ can be expressed as:

$$\begin{aligned} ||\mathbf{y}||_2 &= \sqrt{\sum_{i=1}^{d} (\text{Cov}_{i1} v_1 + \text{Cov}_{i2} v_2 + \cdots + \text{Cov}_{id} v_d)^2} \\ &\le \sqrt{\sum_{i=1}^{d} (\sum_{j=1}^{d} b_i^2 b_j^2) \cdot c^2}. \end{aligned} \tag{16}$$

*Lazy normalization* Based on the upper bound of the Euclidean norm, we design a vector normalization method called *Lazy Normalization* to scale the eigenvectors. Let $\mathbf{y}$ remain the approximate eigenvector obtained from the covariance matrix transformation $\mathbf{y} = \text{Cov} \cdot \mathbf{v}$, and denote the upper bound for its Euclidean norm as $B$. We then set the evaluation interval for the iterative InvSRT algorithm as $(0, B^2]$. As previously mentioned in this section, a reasonably accurate initial approximation within this interval should be identified to reduce the number of rounds executed in the iterative InvSRT. To achieve this, we conduct an odd-degree Taylor expansion $T(x)$ of $f(x) = \frac{1}{\sqrt{x}}$ centered at the midpoint $B^2/2 + 1$. This expansion serves as the initial approximation function for the iterative InvSRT (see Algorithm 9). It has been empirically demonstrated that such a Taylor expansion enables the subsequent iterative InvSRT to converge within a relatively modest number of iterations (Qu and Xu, 2022), as compared to alternative approaches (Gizem et al., 2015), (Panda, 2021), (Panda, 2022). The iterative InvSRT then takes the initial approximation $T(\langle \mathbf{y}, \mathbf{y} \rangle)$ as input and iterate $\tau$ times to obtain a more accurate value of $\frac{1}{||\mathbf{y}||_2}$. The approximate eigenvector is then multiplied by the output of the iterative InvSRT to perform the normalization.

For any $x \in (0, b^4 \cdot c^2 \cdot d^2]$, it holds that $T(x) \le \frac{1}{\sqrt{x}}$ (Qu and Xu, 2022). Furthermore, the iterative InvSRT output is no greater than the ground truth of InvSRT (Theorem 4). Hence, we can estimate that both the Euclidean have an upper bound of 1 after the normalization. This allows us to recompute the upper bound of the Euclidean norm for the next approximate eigenvector $\mathbf{y}' = \text{Cov} \cdot \mathbf{y}$. In particular, if the length of $\mathbf{v}$ is initially 1, then the upper bound of $||\mathbf{y}'||_2$ remains unchanged and equal to the upper bound of $||\mathbf{y}||_2$.

The degree of $T(x)$ is minimized to prevent generating extremely small high-degree coefficients that exceed the precision range of the homomorphic encryption scheme. This may lead to a result that the accuracy of $T(x)$ in the interval $(0, 1)$ is lower compared to the interval $[1, B^2]$, due to the significant difference in slopes on both sides of 1 for $\frac{1}{\sqrt{x}}$. Subsequent iterative InvSRT inherits this disparity of accuracy. It is impractical to achieve the same level of accuracy for the iterative InvSRT across the evaluation interval $(0, 1)$ as in the interval $[1, B^2]$,

since a significant number of iterations is required. The best we can do is to make an assumption that the covariance matrix transformation does not shrink the length of vectors by more than a factor of $S$. Based on this assumption, we can attempt to ensure that (i) the length of the eigenvector does not underflow during the PowerMethod iterations and (ii) the length of the final output eigenvector from the PowerMethod is sufficiently close to 1.

Therefore, our strategy is as follows. Let $B'$ be a precision bound, $e$ be an error bound, and $t$ be the expected number of PowerMethod iterations. Denote $S$ as the contraction coefficient of the covariance matrix transformation, which means that any vector, when transformed by the covariance matrix, is reduced in length by at most a factor of $S$. We then simulate the PowerMethod iterations on a unit-length vector to obtain the best-fit number of iterative InvSRT iterations in each PowerMethod iteration:

1. We start with an initial length $l$ of 1. In each PowerMethod iteration, we first update $l$ directly as $l \leftarrow l \cdot S$, without performing any actual covariance matrix transformation.
2. Subsequently, we evaluate $T(l^2)$ and feed it into the iterative InvSRT. The resulting output, denoted as $S'$, must satisfy the following two conditions: (i) $(l \cdot S' \cdot S)^2$ should not underflow beyond the predetermined precision bound $B'$, and (ii) the value $1 - l \cdot S'$ must be smaller than $e$ in the final iteration of the PowerMethod.
3. $l$ is updated as $l \leftarrow l \cdot S'$ in preparation for the subsequent round of PowerMethod iteration.
4. We maintain a record of the number of iterations employed by the iterative InvSRT during each iteration of the PowerMethod. In the actual homomorphic PowerMethod computation, we rely on these recorded iteration counts to perform the iterative InvSRT.

Up to this point, our Lazy Normalization strategy can be uniquely characterized by its evaluation interval $(0, B^2]$, initial approximation function $T$, precision bound $B'$, error bound $e$, number of PowerMethod iterations $t$, and the number of iterative InvSRT iterations per PowerMethod iteration. Consequently, we can conclude that the Lazy Normalization strategy is capable of accommodating $t$ PowerMethod iterations that involve covariance matrix transformations with a contraction coefficient of $S$ while maintaining an error bound $e$ and a precision bound $B'$.

**Theorem 4.** *If Algorithm 3 is given an input $y_0 \le \frac{1}{\sqrt{x_0}}$, then in any iteration, if $y_{i-1} \le \frac{1}{\sqrt{x_0}}$ and $y_{i-1} > 0$, it follows that $y_{i-1} \le y_i \le x_0$.*

**Proof.** When $y_{i-1} \le \frac{1}{\sqrt{x_0}}$ and $y_{i-1} > 0$, we have $0 < x_0 y_{i-1}^2 \le 1$. Therefore, $y_i \ge y_i$. On the other hand, set $y_{i-1} = x_0 - \epsilon$ where $0 \le \epsilon < \frac{1}{\sqrt{x_0}}$. We can express $y_i$ as follows:

$$\begin{aligned} y_i &= \frac{1}{2}\left(\frac{1}{\sqrt{x_0}} - \epsilon\right)\left(3 - x_0 \cdot \left(\frac{1}{\sqrt{x_0}} - \epsilon\right)^2\right) \\ &= \frac{1}{2}\left(\frac{1}{\sqrt{x_0}} - \epsilon\right)(2 + 2\sqrt{x_0}\epsilon - x_0\epsilon^2) \\ &= \frac{1}{\sqrt{x_0}} - \frac{\sqrt{x_0}\epsilon^2}{2} - \sqrt{x_0}\epsilon^2 + \frac{x_0\epsilon^3}{2} \\ &\le \frac{1}{\sqrt{x_0}}. \end{aligned} \tag{17}$$

□

### 6.1.3. Eigenvalue computation

The process of computing the eigenvalue is similar to completing another round of Power Method iterations. Let $\mathbf{v}$ be the eigenvector obtained after $t$ iterations. The eigenvalue $\lambda$ is given by $\lambda = \frac{\langle \mathrm{Cov}\cdot\mathbf{v}, \mathbf{v}\rangle}{\langle \mathbf{v}, \mathbf{v}\rangle}$. We know that $\mathbf{v}$ has been normalized after $t$ iterations. Therefore, if we are confident enough in the accuracy of the normalization process, the eigenvalue computation can be simplified to $\lambda = \langle \mathrm{Cov}\cdot\mathbf{v}, \mathbf{v}\rangle$. The process of $\mathrm{Cov}\cdot\mathbf{v}$ is exactly the same as the first half of a new round of Power Method iterations. However, $\mathrm{Cov}\cdot\mathbf{v}$ and $\mathbf{v}$ have different orientations, i.e., if one is a column vector, the other is a row vector, and vice versa. Therefore, when performing the dot product, we need to flip the orientation of $\mathbf{v}$ to match that of $\mathrm{Cov}\cdot\mathbf{v}$. The axis flipping algorithm (AxisFlipping) is provided in Algorithm (11). Next, computing $\langle \mathrm{Cov}\cdot\mathbf{v}, \mathbf{v}\rangle$ only requires $k$ ciphertext multiplications and one aggregation operation. We can incorporate the eigenvalue computation into the PowerMethod iteration, specifically in the last round of the Power Method (total $t+1$ rounds), where the iteration is used to compute the eigenvalue instead of updating the eigenvector. Note that we need to retain $\mathbf{v}^T$ to compute the covariance matrix after the eigenvalue transformation.

**Algorithm 9** TaylorInit.

**Input:** $ct(x)$: ciphertext encrypting value $x$; $B$: upper bound for the iterative InvSRT evaluation interval; $o$: truncation order.
**Output:** $ct(x')$, where $x' = T(x)$, and $T(x)$ is the Taylor expansion polynomial of $\frac{1}{\sqrt{x}}$ at point $B/2+1$ with truncated order $o$.

**Algorithm 10** IterativInvSRTbyNewton.

**Input:** $ct(x)$: ciphertext encrypting point; $x$ $ct$(Guess): initial approximation of iterative InvSRT at point $x$; $\tau$: number of iterations.
**Output:** $ct(x'), x' \approx \frac{1}{\sqrt{x}}$

```
ct(x') ← ct(Guess)
for i range 0 to τ − 1 do
    ct(Temp) ← Mult(Mult(ct(x'), ct(x')), Mult(ct(x'), ct(x)))
    ct(Temp) ← Sub(ct(Temp), PMult(pt(3), ct(x')))
    ct(Temp) ← PMult(pt(1/2), ct(Temp))
    ct(x') ← ct(Temp)
end for
return ct(x')
```

**Algorithm 11** AxisFlipping.

**Input:**
$\{ct(V[i]) | 0 \le i < k\}$: a set of encrypted matrices representing one replicated vector; axis: the axis of the vector.
**Output:**
$\{ct(V[i]^T) | 0 \le i < k\}$

```
Mask ← I
for i range 0 to k − 1 do
    ct(V[i]^T) ← PMult(Mask, ct(V[i]))
    ct(V[i]^T) ← Aggregate(ct(V[i]^T); axis + 1 mod 2)
end for
return {ct(V[i]^T) | 0 ≤ i < k}
```

So far, we have discussed the three modules of the Power Method algorithm, namely the linear transformation of the covariance matrix, normalization of the eigenvectors, and computation of the eigenvalues. These modules cover all the processing details of the Power Method, and the complete process is provided in Algorithm 13, where the invoked iterative InvSRT algorithm is implemented by Newton's method with a Taylor expansion as the initial approximation (see Algorithm 9 and 10). We analyze the multiplication depth and the computation complexity of Algorithm 13 in Theorem 5.

**Theorem 5.** *The multiplication depth of Algorithm 13 is $O(l_P \cdot (3 + log(order) + 3 \cdot l_N + 1) + 3)$, and the time complexity is dominated by $O((l_P+1)\cdot(k^2+k+c+3\cdot l_N+1) - 3\cdot l_N - 1)$ ciphertext multiplications and $O(\frac{3k+3}{2}\log n \cdot (l_P+1))$ ciphertext rotations, where $c$ is the number of ciphertext multiplications needed for the Taylor polynomial evaluation.*

**Proof.** In each round of the PowerMethod, a maximum of 3 multiplication depths, $k^2 + k$ ciphertext multiplications, and $(2k+1)\cdot\log n$ (or

**Algorithm 12** Homomorphic EigenShift (HEigenShift).

**Input:**
$ct(\lambda)$, $\{ct(V[i])|0 \le i < k\}$: the dominant eigenvalue and the corresponding eigenvector;
$\{ct(V[i]^T)|0 \le i < k\}$: the transposed eigenvector;
$\{\text{ct}(\text{Cov}_i[j])|0 \le i, j < k\}$: the covariance matrix;
**Output:**
$\{\text{ct}(\text{SCov}_i[j])|0 \le i, j < k\}$: the 1-shifted version of Cov.
**for** $0 \le j < k$ **do**
  **for** $0 \le i < k$ **do**
    $\text{ct}(\lambda \cdot V^T V_i[j]) \leftarrow \text{Mul}(ct(\lambda), \text{Mul}(\text{ct}(V[j]^T), \text{ct}(V[i])))$
    $\text{ct}(\text{SCov}_i[j]) \leftarrow \text{Sub}(ct(\text{Cov}_i[j]), ct(\lambda \cdot V^T V_i[j]))$
  **end for**
**end for**
return $\{\text{ct}(\text{SCov}_i[j])|0 \le i, j < k\}$

$(k+2) \cdot \log n)$ ciphertext rotations are required to accomplish the covariance matrix transformation and one inner product. Subsequently, $\log(order)$ depths and $c$ ciphertext multiplications are needed for the Taylor Expansion. Once we enter Newton's Method iterative algorithm, each iteration necessitates 3 depths and 3 ciphertext multiplications. Lastly, a single multiplication depth is required to perform the ciphertext multiplication for scaling the approximate eigenvector. After conducting $l_P$ rounds of PowerMethod iterations, an additional covariance matrix transformation and inner product are required to complete the eigenvalue calculation. □

### 6.2. Privacy-preserving PCA

The final step is to combine the Power Method with the EigenShift algorithm to compute multiple dominant eigenvectors and their corresponding eigenvalues. After each execution of the Power Method, we replace the covariance matrix with its 1 shifted version, thereby ensuring that for $1 \le k$, the $k$-th round of the Power Method yields the $k$-th dominant eigenvector of the original covariance matrix of the dataset. The implementation of the homomorphic 1-th EigenShift algorithm is presented in Algorithm 12. It utilizes the dominant eigenvalue $\lambda$ and the corresponding eigenvector and its transpose $\mathbf{v}, \mathbf{v}^T$ obtained in the previous Power Method iteration to update the covariance matrix as $\text{Cov}' = \text{Cov} - \lambda \cdot \mathbf{v}^T\mathbf{v}$. It can be proven that the 1-shifted covariance matrix always has a smaller amplification effect on the vectors than its non-shifted version (see Appendix B for details). Thereby, we do not need to recompute the upper bound for the norm of vectors transformed by the shifted covariance matrix in each execution of PowerMethod. The entire process of the privacy-preserving PCA algorithm is provided in Algorithm 14.

One more important consideration is the need for modulus refreshes when the multiplication depth of our algorithm exceeds the maximum multiplication depth supported by the underlying CKKS scheme. It is always necessary to carefully select appropriate positions in the algorithm to perform modulus refreshes in order to minimize the overall number of refresh operations. We defer the discussion on the timing of the modulus refresh operations in our privacy-preserving PCA algorithm to Appendix C for further details.

### 6.3. Security analysis

**Theorem 6.** *The proposed Privacy-Preserving PCA securely and correctly computes the target approximate dominant eigenvectors and eigenvalues of the private dataset in the semi-honest security model, provided that the underlying homomorphic encryption scheme is IND-CPA secure.*

**Proof.** Let the encryption and decryption routines of the underlying homomorphic encryption scheme be the $Enc$ and $Dec$ functions in the proposed system architecture (see Section 3.2). Given that the homomorphic encryption is IND-CPA secure, the ciphertext $ct(X)$ holding the private dataset $X$ from the user looks pseudorandom to the cloud. At any point during the execution of the privacy-preserving PCA protocol, the user only decrypts the received ciphertexts locally. So the cloud

**Algorithm 13** Homomorphic PowerMethod (HPowerMethod).

**Input:**
$\{\text{ct}(\text{Cov}_i[j])|0 \le i, j < k\}$: the encrypted covariance matrix;
$l_P$: iterations for the PowerMethod;
$l_N$: iterations for the InvSRTByNewton;
$\{ct(V_0[i])|0 \le i < k\}$: encrypted submatrices representing the initial selection for the approximated eigenvector;
axis: the axis of the current approximate eigenvectors, where axis = 0 represents row vectors with all rows being equal and axis = 1 represents column vectors with all columns being equal;
$B$: upper bound for the iterative InvSRT evaluation interval;
$order$: truncation order for the Taylor initialization.
**Output:**
$ct(\lambda)$, $\{ct(V_{l_P-1}[i]^T)|0 \le i < k\}$, $\{ct(V_{l_P-1}[i])|0 \le i < k\}$: Dominant eigenvalue of the covariance matrix, the corresponding eigenvector, along with its transposition.
**for** $t$ range 0 to $l_P$ **do**
  **for** i range 0 to k-1 **do**
    $ct(\text{Sum}) \leftarrow ct(\mathbf{0})$
    **for** $j = 0; j < k; j++$ **do**
      **if** axis == 0 **then**
        $ct(\text{Sum}) \leftarrow \text{Add}(\text{Mult}(ct(\text{Cov}_j[i]), ct(V_t[i])), ct(\text{Sum}))$
      **else if** axis == 1 **then**
        $ct(\text{Sum}) \leftarrow \text{Add}(\text{Mult}(ct(\text{Cov}_i[j]), ct(V_t[i])), ct(\text{Sum}))$
      **end if**
    **end for**
    $ct(V_{t+1}[i]) \leftarrow \text{Aggregate}(ct(\text{Sum}); \text{axis} + 1 \bmod 2)$
  **end for**
  axis $\leftarrow$ axis + 1 mod 2
  $ct(\text{Sum}) \leftarrow ct(\mathbf{0})$
  **for** i range 0 to k-1 **do**
    $ct(\text{Sum}) \leftarrow \text{Add}(\text{Mult}(ct(V_{t+1}[i]), ct(V_{t+1}[i])), ct(\text{Sum}))$
  **end for**
  $ct(\text{Sum}) \leftarrow \text{Aggregate}(ct(\text{Sum}); \text{axis} + 1 \bmod 2)$
  **if** $n < l_P$ **then**
    $ct(\text{Guess}) \leftarrow \text{TaylorInit}(ct(\text{Sum}), B, order)$
    $ct(\text{Sum}) \leftarrow \text{IterativInvSRTbyNewton}(ct(\text{Guess}), ct\text{Sum}, l_N)$
    **for** i range 0 to k-1 **do**
      $ct(V_{t+1}[i]) \leftarrow \text{Mult}(ct(V_{t+1}[i]), ct(\text{Sum}))$
    **end for**
  **end if**
  **if** $t = l_P$ **then**
    $\{ct(V_t[i]^T)|0 \le i < k\} \leftarrow \text{AxisFlipping}(\{ct(V_t[i])|0 \le i < k\}, \text{axis} + 1 \bmod 2)$
    $ct(\text{Sum}) \leftarrow ct(\mathbf{0})$
    **for** i range 0 to k-1 **do**
      $ct(\text{Sum}) \leftarrow \text{Add}(\text{Mult}(ct(V_{t+1}[i]), ct(V_t[i])), ct(\text{Sum}))$
    **end for**
    $ct(\lambda) \leftarrow \text{Aggregate}(ct(\text{Sum}); \text{axis} + 1 \bmod 2)$
  **end if**
**end for**
return $ct(\lambda)$, $\{ct(V_{l_P-1}[i]^T)|0 \le i < k\}$, $\{ct(V_{l_P-1}[i])|0 \le i < k\}$

**Algorithm 14** Privacy-Preserving PCA.

**Input:**
$\{\text{ct}(X_j[\ell])|0 \le j < k, 0 \le \ell < \lceil s/n \rceil\}$: partitioned and encrypted dataset;
$n_Z$: maximum diagonal index for transformation $Z$;
$n_T$: maximum diagonal index $n_T$ for transformation $T$;
$l_E$: expected number of eigenvectors or principal components. $l_P$: iterations for the PowerMethod;
$l_N$: iterations for the InvSRTByNewton;
$B$: upper bound for the iterative InvSRT evaluation interval;
$order$: truncation order for the Taylor initialization.
**Output:**
$\{\{ct(V[i])|0 \le i < k\}_j|0 \le j < l_E\}$, $\{ct(\lambda_j)|0 \le j < l_E\}$: $l_E$ dominant eigenvectors and the corresponding eigenvalues.
$\{ct(\text{Cov}_j[\ell])\} \leftarrow \text{HCovMtrx}(\text{ct}(X_j[\ell])|0 \le j < k, 0 \le \ell < \lceil s/n \rceil, n_Z, n_T)$
**for** $m := 0; m < l_E; m++$ **do**
  $\{ct(V_0[i])|0 \le i < k\} \leftarrow$ Randomly Generate with $\ell_2 = 1$
  axis $\leftarrow$ 1
  $ct(\lambda_m), \{ct(V)\}_m, \{ct(V^T)\}_m, \leftarrow$ HPowerMethod($\{ct(\text{Cov}_j[\ell])\}, l_P, l_N, \{ct(V_0[i])\}$, axis, $B$, order)
  $\{ct(\text{Cov}_j[\ell])\} \leftarrow \text{HEigenShift}(ct(\lambda_m), \{ct(V)\}_m, \{ct(V^T)\}_m, \{ct(\text{Cov}_j[\ell])\})$
**end for**
return $\{\{ct(V[i])|0 \le i < k\}_j|0 \le j < l_E\}$, $\{ct(\lambda_j)|0 \le j < l_E\}$

**Table 3**
Run-time performance Comparison of LinTrans $Z, T$ between original double-hoisting BSGS(dh-BSGS) and our Diagonal Convergence Decomposed scheme (DCDmp). **Rtk(MB)** represents the total space required for rotation keys, while **Ct(MB)** represents the total space required for caching hoisted ciphertexts during linear transformations, both measured in MB (megabytes).

| Scheme | Matrix | $n_1$ | MaxDiagNo. | Rtk (MB) | Ct (MB) | time (s) | Depth |
|---|---|---|---|---|---|---|---|
| dh-BSGS | Z | 8 | 127 | 1755 | 53 | 2.004 | 1 |
| DCDmp | Z | 8 | 64 | 1080 | 53 | 1.362 | 2 |
| DCDmp | Z | 8 | 32 | **720** | 53 | 1.249 | 4 |
| dh-BSGS | Z | 16 | 127 | 1395 | 113 | 1.286 | 1 |
| DCDmp | Z | 16 | 64 | 1080 | 113 | **1.094** | 2 |
| DCDmp | Z | 16 | 32 | 900 | 113 | 1.179 | 4 |
| dh-BSGS | T | 8 | $127 \cdot 128$ | 1035 | 53 | 1.049 | 1 |
| DCDmp | T | 8 | $64 \cdot 128$ | 720 | 53 | 0.701 | 2 |
| DCDmp | T | 8 | $32 \cdot 128$ | **540** | 53 | 0.652 | 4 |
| dh-BSGS | T | 16 | $127 \cdot 128$ | 1035 | 113 | 0.725 | 1 |
| DCDmp | T | 16 | $64 \cdot 128$ | 900 | 113 | **0.572** | 2 |
| DCDmp | T | 16 | $32 \cdot 128$ | 810 | 113 | 0.613 | 4 |

only learns the parameters for initializing the protocol, and nothing more. □

## 7. Implementation

We implement all the proposed algorithms described in the previous sections using the Lattigo library V4.1.0 (Lattigo, 2022), which provides an implementation of the full-RNS CKKS scheme in Golang. The implementation of all the algorithms can be found in Ma (2023). Our experiments are conducted on a machine running Windows 10, equipped with an AMD Ryzen 7 7735HS (3.19 GHz) processor and 32 GB of memory.

First, we present the performance results of the $Z$ and $T$ linear transformations using our diagonal convergence decomposition technique proposed in Section 4. Then, we proceed to report the performance of our privacy-preserving PCA and compare it with the state-of-art work by Panda (Panda, 2021). Their work is the most relevant to our scenario, as it also focuses on designing privacy-preserving PCA based on the CKKS scheme in a cloud computing service setting.

It's worth noting that the pioneering works by Rathee et al. and Lu et al. are also relevant to the cloud computing scenario (Lu et al., 2016), (Rathee et al., 2018). However, as explained in Related Work (see Section 2.1), their adoption of a homomorphic encryption scheme that only permits integer operations necessitates users to compute vector normalization on their own. This disparity makes a direct comparison between our approach and theirs challenging, given our emphasis on minimizing user computational load in the protocol, wherein users refrain from executing any operations beyond protocol initialization, encryption, and decryption.

### 7.1. Performance of $Z$ and $T$ using diagonal convergence decomposition

We conduct tests on the space optimization effect of our Diagonal Convergence Decomposition on the $128 \times 128$ double-hoisting matrix multiplication. We evaluate the performance of the $Z$ and $T$ transformations decomposed at different maximum expected diagonal indices and compare them with the original double-hoisting ones with no decomposition applied (see Table 3). The evaluation is conducted on a concrete CKKS instance that offers a maximum available modulus level of 14 with parameters: $\log N = 15, \log b = 760, \sigma = 3.2$ (see Appendix A for detailed parameter explanation). A baseline level of $14 - 4 = 10$ is set, and the initial modulus levels for all tested transformations are assigned appropriately to ensure that all transformation results fall down to the baseline level. From Table 3, we can observe that for a fixed inner loop count, a smaller expected maximum diagonal index leads to a greater reduction in key space. In the decomposition with an expected maximum diagonal index four times smaller than 128, we successfully reduce the rotation key space of $Z$ by 58% for an inner loop of 8 and by 35% for an inner loop of 16.

Additionally, our decomposition improves the computation speed of $Z$ and $T$. This is because, within the matrix transformation product chain obtained after decomposition, the transformations that are initially performed at high levels (the ones on the right-hand side of the chain) require only a small amount of complexity, and the main complexity arises from the final matrix transformation in the chain (the one on the left-hand side), which is performed at the same level as the original scheme and has a smaller scale compared to the transformation that was not decomposed. Therefore, our decomposition scheme provides an optimized trade-off between space and speed, making the overall matrix multiplication scheme more flexible.

### 7.2. Privacy-preserving PCA

#### 7.2.1. Accuracy measurement

We provide a metric called $R2(V)$ to measure the accuracy of the privacy-preserving PCA algorithm. It represents the R2 score between the results computed by privacy-preserving PCA and the ground truth principal components computed using the *np.linalg.eig* function from the *numpy* package in Python. A higher value of $R2(V)$, closer to 1, indicates that the principal components obtained from privacy-preserving PCA are more similar to the ones obtained using plaintext routines.

#### 7.2.2. Experimental setup

*Modulus refresh strategy selection* The modulus refresh operation in our privacy-preserving PCA algorithm can be theoretically achieved through the following two approaches based on the cloud service scenario.

1. The interactive strategy: the cloud service provider sends the ciphertexts that have reached the maximum depth to the user. The user then decrypts these ciphertexts, re-encrypts them on the maximum modulus level, and sends them back to the cloud for further homomorphic computation. This approach introduces communication overhead and requires the user to listen to the cloud continuously.
2. The non-interactive strategy: the cloud service provider performs bootstrapping locally on the ciphertexts. While several bootstrapping methods designed for CKKS have been proposed in recent years, their software implementations are still relatively time-consuming. Therefore, the number of modulus refreshes will become one of the dominant factors in the algorithm's time complexity when using this strategy.

We use the interactive modulus refresh strategy throughout our experiment since our algorithm consumes a large number of modulus

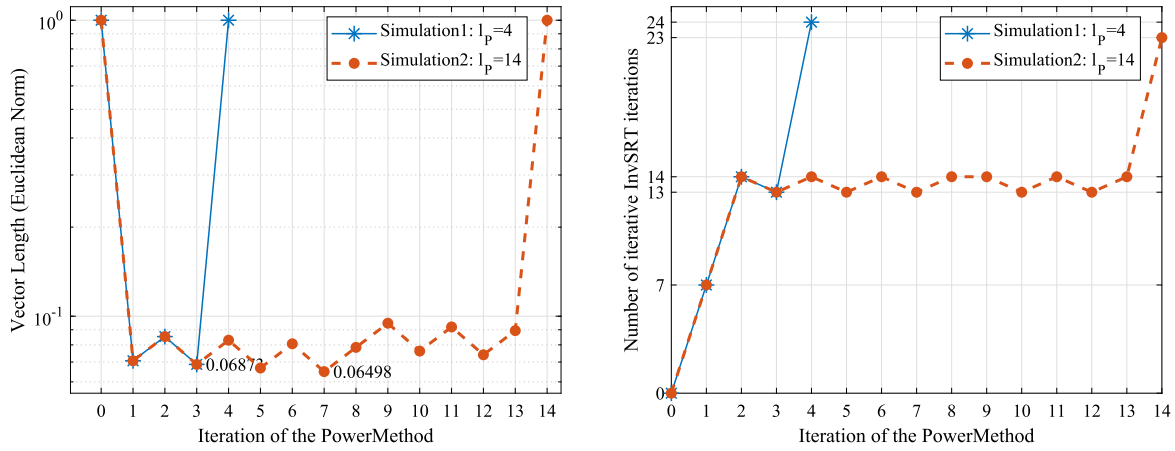


**Fig. 2.** PowerMethod Simulations for Vector Normalization Parameterization. The figures presented on the left and right respectively illustrate the evolution of vector lengths and the number of iterative InvSRT iterations per PowerMethod iteration in 4 and 14 rounds of simulations. The simulations are conducted with an error bound of $e = 1e-5$ and a precision bound of $p = 1e-3$.

levels and does not yield satisfactory results when the bootstrapping routine is utilized to achieve the non-iterative modulus refresh strategy. However, it is worth noting that there exist hardware-accelerated solutions for bootstrapping that can significantly reduce its computational overhead. Our privacy-preserving PCA algorithm may become truly practical under the non-interactive modulus refresh strategy with such hardware optimizations.

*Parameterization for underlying homomorphic encryption scheme* We set up a concrete CKKS instance with parameters $\log N = 15, b = 720, \sigma = 3.2, \delta = 40$ as the underlying homomorphic encryption scheme of our privacy-preserving PCA algorithm (see Appendix A for the explanation of parameters), ensuring 128-bit security. This instance provides 13 levels of 40-bit moduli and achieves 20 bits of precision.

*Dataset preprocessing* We conduct our privacy-preserving PCA algorithm on the MNIST (Deng, 2012), Fashion-MNIST (Xiao et al., 2017), and YALE (Belhumeur et al., 1997) datasets. The preprocessing of the three datasets, including cropping, is performed exactly as described in the work by Panda (Panda, 2021). The images in the MNIST and Fashion-MNIST datasets are cropped to 16x16 pixels, and the Yale samples are converted to grayscale and cropped from 195 x 231 pixels to 16x16 pixels. In the subsequent experiments, we will extract a specific number of samples $s$ from these datasets to construct new datasets for testing our algorithm. All the datasets used in the experiments are divided into submatrices of size $128 \times 128$ since our underlying homomorphic encryption scheme offers a plaintext space of $\mathbb{C}^{14}$ where each ciphertext can accommodate exactly one submatrix.

*Parameterization for covariance matrix computation* Our covariance matrix computation is designed to harness the advantages of parallel matrix multiplications. We employ a single thread to compute the covariance matrix in cases where the dataset size is no larger than $2 \times 2$ sub-matrices. As for dataset sizes surpassing this threshold, we allocate seven threads to compute the corresponding covariance matrix. The $128 \times 128$ matrix multiplications within the covariance matrix computation are implemented by our optimized matrix multiplication algorithm equipped with the hoisting techniques and a Diagonal Convergence Decomposition strategy with an expected maximum non-zero diagonal index of 32. This configuration reduces 35% and 20% in the requisite number of rotation keys for the $Z$ and $T$ transformations with an inner loop count of 16, respectively. Such optimization mitigates the risk of memory saturation, diminishes the communication overhead between the user and the cloud, and decreases computational time. We also implement all three covariance matrix computation optimizations mentioned in Section 5.2. Specifically, we group every 1400 rows of samples into one segment and apply optimizations 1 and 2 to each segment when the dataset contains more than 1400 rows. The covariance matrices computed from different segments are aggregated to obtain the final result.

*Parameterization for vector normalization* In section 6.1.2, we estimate that performing a covariance matrix transformation on a vector of length $c$ results in a transformed vector with an upper bound length of $b^2 \cdot c \cdot d$, where $b$ is the upper bound of all elements in the covariance matrix and $d$ is the number of features in the dataset. Since the upper bound length of the transformed vector will be used to set up the evaluation interval of the iterative InvSRT function, we should not let this upper bound be too big or it will be too expensive to evaluate the InvSRT function across the interval. Thereby, we scale all samples in the preprocessed datasets by a factor of $1/255$, resulting in an upper bound $b = 1$ for all elements in their covariance matrices.

Following the Lazy Normalization strategy, we set the evaluation interval of the iterative InvSRT function to $[0.1^{-7}, b^2 \cdot c \cdot d]$ and use a linear Taylor expansion at point $2^{16}/2 + 1$ to obtain the initial approximation of the iterative InvSRT algorithm. We determine through experimentation that the iterative InvSRT requires a minimum of $k = 12$ basic iterations to achieve the desired accuracy within the evaluation interval $[1, 2^{16}]$. In order to make the Lazy Normalization strategy capable of accommodating up to $l_P$ PowerMethod iterations that involve covariance matrix transformations with a contraction coefficient of 0.5 while maintaining an error bound $e = 1e-5$ and a precision bound $p = 1e-3$, PowerMethod simulations are conducted to select the number of iterative InvSRT iterations in each PowerMethod iteration, for $l_P = 4$ and $l_P = 14$ (see Fig. 2). Consequently, we manage to employ an average of 14.5 iterative InvSRT iterations per PowerMethod iteration for $l_P = 4$, and an average of 13.8 iterative InvSRT iterations per PowerMethod iteration for $l_P = 14$.

The vector normalization strategy in the work of Panda is fixed with an evaluation interval of $[0.001, 750]$ and a linear initial approximation function of $y = -0.00019703x + 0.14777278$ for the iterative InvSRT. Only 6 iterations of InvSRT are employed in each PowerMethod iteration. If we evaluate the effectiveness of their vector normalization strategy using the PowerMethod Simulation we employ in Lazy Normalization, their strategy only accommodates 4 PowerMethod iterations for an error bound of $1e-2$ involving covariance matrix transformations with a contraction coefficient of 1. Note that the dataset pre-scaling in the work by Panda is set differently based on the size of the samples in the datasets in order to match the fixed evaluation interval of vector normalization. However, such an approach is impractical in real-world

**Table 4**
Performance Comparison between privacy-preserving PCA algorithms in Panda (2021) and Ours under interactive modulus refresh strategy. $l_E$ denotes the desired number of principal components, $l_P$ represents the number of iterations for the PowerMethod, $l_N$ represents the average number of iterative InvSRT, $Lvs/PM$ represents the average modulus levels consumed per PowerMethod iteration, and *time* denotes the time for the entire computation, where our approach's time consists of two parts: the computation time of the covariance matrix (left) and the computation of principal components, i.e., the PowerMethod and EigenShift algorithms (right).

| Scheme | DataSet | $s$ | $l_E$ | $l_P$ | $l_N$ | *time* (min) | $R2(V)$ | Lvs/PM |
|---|---|---|---|---|---|---|---|---|
| (Panda, 2021) | MNIST | 200 | 4 | 4 | 6 | 5.21 | 0.047 | 77 |
| Ours | MNIST | 200 | 4 | 4 | 14.5 | 0.56 + 2.16 | 0.174 | 61 |
| (Panda, 2021) | MNIST | 200 | 4 | 14 | 6 | 18.42 | 0.392 | 77 |
| Ours | MNIST | 200 | 4 | 14 | 13.8 | 0.58 + 8.41 | 0.388 | 54 |
| (Panda, 2021) | MNIST | 60000 | 8 | 14 | 6 | 1628 | 0.445 | 77 |
| Ours | MNIST | 60000 | 8 | 14 | 13.8 | 45+14 | 0.641 | 54 |
| (Panda, 2021) | F-MNIST | 200 | 4 | 4 | 6 | 5.3 | 0.600 | 77 |
| Ours | F-MNIST | 200 | 4 | 4 | 14.5 | 0.58 + 2.23 | 0.633 | 61 |
| (Panda, 2021) | F-MNIST | 60000 | 8 | 14 | 6 | 1628 | 0.626 | 77 |
| Ours | F-MNIST | 60000 | 8 | 14 | 13.8 | 45 + 14 | 0.911 | 54 |
| (Panda, 2021) | Yale | 165 | 6 | 14 | 6 | 27.5 | 0.715 | 77 |
| Ours | Yale | 165 | 6 | 14 | 13.8 | 0.62 + 11 | 0.744 | 54 |

scenarios where the user with little knowledge about the internal detail of the algorithm may perform the pre-scaling inappropriately, posing a high probability of overflow (or underflow).

To make a fair comparison with our vector normalization approach, we fix the dataset scaling factor as $1/255$ for the work by Panda in our experiment instead of changing it across different datasets. This choice is close to the scaling factor used in their experiments on the Fashion MNIST dataset with 200 samples.

### 7.2.3. Performance comparison

Based on the aforementioned experimental setup, we conduct our privacy-preserving PCA algorithm and compare its performance with the previous work of Panda. (Panda, 2021). Table 4 presents the performance of our privacy-preserving PCA algorithm and the algorithm proposed in Panda (2021)., both using the interactive modulus refresh strategy.

*Efficiency and scalability* We can observe that for the same number of principal components and PowerMethod iterations, our algorithm is approximately 1.9 times faster in terms of the overall runtime. Specifically, our PowerMethod runs at a speed of approximately 8.2 seconds per iteration, which is around 2.3 times faster than the 19.6 seconds reported by Panda per iteration. This notable improvement can be attributed to a lower number of ciphertext rotations achieved by our PowerMethod (see Theorem 5) compared to their reported complexity of $O(l_P \cdot (2\log d \cdot \frac{s \cdot d}{N/2} + \log \frac{N/2}{d}))$ (where $s$ remains the number of samples and $d$ the number of features). This advantage becomes more apparent when dealing with larger sample sizes. As an example, we are able to compute the top 8 eigenvectors of a dataset with 60,000 samples in approximately 1 hour using 7 threads, which is estimated to be roughly 25 times faster than the work by Panda using the same number of threads. This underscores the scalability of our approach: When faced with datasets containing a large number of samples, the complexity of our solution primarily concentrates on homomorphic covariance matrix computation, a one-time operation. The existence of the covariance matrix enables the subsequent PowerMethod execution time entirely independent of the number of samples, which implies that the user may apply more rounds of PowerMethod to the computed principal components at any time if he or she is unsatisfied with the precision of the principal components. Furthermore, it is worth noting that if a user wishes to augment the dataset with new samples and compute the principal components of the new dataset using privacy-preserving PCA, the complexity of this operation is also independent of the original dataset size. This is because, for any dataset $X$, we can easily generate the new covariance matrix by aggregating the precomputed covariance matrix of the original and the one generated by the new samples with equation $Cov' = 1/(|X| + |X'|) \cdot (X^T X + X'^T X') - \mu' \mu'^T$, where $X'$ is the set consists of all new samples and $\mu'$ is the average vector of the whole dataset.

*Accuracy* Enhancements in $R2(V)$ accuracy are recorded in MNIST and FMNIST datasets, each containing 60000 samples. This improvement is attributed to a specific factor: The approximate eigenvector in each PowerMethod iteration has a higher probability of being precisely normalized due to our larger evaluation interval for vector normalization (encompassing the maximum input value and a closer-to-zero left boundary) and more rounds of iterative InvSRT, compared to the approach employed by Panda. On the other hand, no discernible enhancements in accuracy are observed in experiments on YALE (165 samples), Fashion-MNIST (200 samples), and MNIST (200 samples, 14 PowerMethod iterations), implying situations that the inputs of vector normalization happen to fall into the intersection of the evaluation intervals prepared by both Panda and our scheme.

From a different perspective, the variations in accuracy among different datasets mainly stem from the inherent distribution of eigenvalues in each dataset (von Mises and Pollaczek-Geiringer, 1929). Specifically, the Fashion MNIST dataset exhibits higher accuracy due to the relatively larger differences among its dominant eigenvalues. For example, its eight dominant eigenvalues are [6.50301, 4.06187, 1.39494, 1.13345, 0.93763, 0.81907, 0.49591, 0.45021] compared to those with smaller differences in the MNIST dataset, namely [1.80816, 1.24723, 1.08723, 0.98732, 0.85060, 0.74359, 0.63526, 0.57545].

*Modulus utilization* Additionally, our modulus utilization is higher than the approach proposed by Panda. For a total of 4 PowerMethod iterations, our approach requires an average of 61 modulus levels for each PowerMethod iteration with an average of 14.5 iterative InvSRT iterations, while their approach requires an average of 77 modulus levels to complete one PowerMethod iteration with only 6 iterative InvSRT iterations. This discrepancy arises due to two factors: (i) We composite two iterative InvSRT iterations into one and employ more ciphertext multiplications to save 1 modulus level per iterative InvSRT iteration. (ii) their PowerMethod implementation has a multiplication depth that is dependent on the dataset's number of features. In contrast, our approach's multiplication depth is independent of the dataset's features. As a result, we have more flexibility in performing additional rounds of the iterative InvSRT algorithm to achieve a more accurate vector normalization.

## 8. Conclusion and further discussion

In this paper, we have presented a novel and efficient privacy-preserving PCA scheme using homomorphic in the context of cloud computing, which has addressed several challenges presented in previous approaches. Firstly, we have successfully tackled the obstacle of homomorphically computing the covariance matrix, a limitation in previous methods. This is achieved by designing an efficient homomorphic covariance calculation algorithm, leveraging our optimized matrix multiplication as a core component, and utilizing parallel computations of multiple matrix multiplication instances to enhance speed. Secondly, we have introduced an efficient homomorphic circuit for the PowerMethod algorithm, which incorporates a universal vector normalization strategy to address the issue of potential accuracy loss. The experimental results demonstrate that our approach surpasses state-of-the-art methods, offering reduced runtime, improved accuracy, and enhanced scalability.

There are several topics for further discussion, including but not limited to:

1. A significant portion of the complexity in homomorphic matrix multiplication comes from row and column transformations. Recent work by Jang et al. introduced optimizations for matrix multiplication that effectively reduce the complexity of these transformations (Jang et al., 2022). Their optimizations could be complementary to ours, and combining them may lead to even better performance.
2. The performance of PowerMethod depends on the characteristics of the dataset itself, especially the distribution of eigenvalues. If the eigenvalues of the covariance matrix are close to each other, the accuracy of PowerMethod may be reduced (von Mises and Pollaczek-Geiringer, 1929). It may be worth considering alternative PCA algorithms to achieve more accurate results. Notably, our homomorphic covariance matrix computation algorithm can serve as a black box function for alternative privacy-preserving PCA algorithm design.
3. Our proposed solution can serve as a candidate for multi-party privacy-preserving PCA. Specifically, our approach can be migrated to multi-party homomorphic encryption such as the work proposed by Mouchet et al. (2021) and provide possible solutions for both vertical and horizontal federated privacy-preserving PCA scenarios.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

Data will be made available on request.

## Acknowledgements

This research is supported by the National Key Research and Development Program of China 2021YFF0704102, and the National Natural Science Foundation of China (62076125).

## Appendix A. Detailed information of full-RNS CKKS scheme

We use $\Phi_M(X)$ to denote the $M$-th cyclotomic polynomial and denote its $N = \varphi(M)$ roots as $\zeta_1, \zeta_3, \ldots, \zeta_{\varphi(M)}$, where $\zeta_i = \zeta_1^i$. The symbol $R$ represents the cyclotomic polynomial ring $\mathbb{Z}[X]/\Phi_M(X)$. Let $Q_L = q_0 \times q_1 \times \cdots \times q_L$ be the product of prime numbers $q_i$. The elements in ring $R_{Q_L}$ can be uniquely represented in the RNS field as $R_{q_0} \times R_{q_1} \times \cdots \times R_{q_L} \cong R_{Q_L}$. The introduction of full-RNS CKKS can be divided into the following four modules.

### A.1. General module

This module includes the parameterization for full-RNS CKKS, the encoding method to convert a complex vector to a native plaintext form, the encryption method to convert plaintext to ciphertext, as well as the key switching technique to change the decryption key associated with a specific ciphertext.

**Parameter Generation** Setparams$(N, b, \sigma)$ sets $M = 2N$ and selects the $M$-th cyclotomic polynomial $\Phi_M(X) = X^N + 1$. Two sets of prime numbers $\{q_i | i = 0, \ldots, L\}$ and $\{p_i | i = 0, \ldots, \alpha - 1\}$ are chosen such that their products are at most $b$ bits. These primes satisfy $q_i, p_i \equiv 1 \bmod 2N$. Let $Q_L = \prod_{i=0}^{L} q_i$ and $P = \prod_{i=0}^{\alpha} p_i$. The ring $R_{Q_L} = \mathbb{Z}[X]_{Q_L}/\Phi_M(X)$ is referred to as the plaintext space of the CKKS scheme. The distribution over $R$ for generating the secret key is denoted as $R \leftarrow \chi_s$, and the error distribution over $R$ as $R \leftarrow \chi_e$, which is a truncated discrete Gaussian distribution with standard deviation $\sigma$.

**Encoding:** Encode$(\mathbf{z}, \Delta, n, \ell)$ takes a message vector $\mathbf{z} \in \mathbb{C}^n$, where $1 \leq n < N$ and $n$ divides $N$, and converts it to a plaintext polynomial $m \in R_{Q_\ell}$. To prevent a significant precision loss in the conversion, a scaling factor $\Delta$ is applied to the message. For any plaintext in $R_{Q_\ell}$, denote $\ell$ as its modulus level.

**Decoding:** Decode$(m, \Delta, n, \ell)$ takes a plaintext polynomial $m \in R_{Q_\ell}$ as input and converts it to a message vector $\mathbf{z} \in \mathbb{C}^n$, where $1 \leq n < N$ and $n$ divides $N$. The decoding process is the inverse operation of encoding.

**Secret Key Generation:** SecKeyGen$(\cdot)$ samples $s \leftarrow \chi_s$ and outputs the secret key $s$.

**Public Key Generation:** PubKeyGen$(s)$ samples $e \leftarrow \chi_e$ and $a \in_u R_{Q_L}$, and outputs $(-as + e, a)$.

**Encryption:** Enc$(m, pk)$ takes a plaintext polynomial $m \in R_{Q_\ell}$ and a public key $pk \in R_{Q_L}^2$ as input. It samples $a \in R_{Q_\ell}$ and outputs $(a \cdot pk_0, a \cdot pk_1) + (m + e_0, e_1) \in R_{Q_\ell}^2$ as a ciphertext encrypting $m$. For any ciphertext in $R_{Q_\ell}^2$, denote $\ell$ as its own modulus level.

**Decryption:** Dec$(c, sk)$ takes a ciphertext $c \in R_q^2$ and a secret key $sk$ as input and outputs a plaintext polynomial $m' \in R$. It computes and outputs $m' = c_0 + c_1 \cdot sk \in R_{Q_\ell}$.

**Switch Key Generation:** SwitchKeyGen$(s, s', \mathbf{b})$ for $\mathbf{b} \in \mathbb{Z}^\beta$ a vector representation of a certain integer decomposition basis, SwitchKeyGen takes in $s, s'$ and outputs a vector as the switching key: $swk_{s,s'} = (swk_{s,s'}^{(0)}, \ldots, swk_{s,s'}^{(\beta-1)})$ where $swk_{s,s'}^{(i)} \in R_{PQ_L}^2$.

**Key Switching:** KeySwitch$(c, swk_{s,s'})$ takes a polynomial $c = (a, b) \in R_{Q_\ell}^2$ and a switching key $swk_{s,s'}$ as input. It first decomposes $a$ with respect to the decomposition basis $\mathbf{b}$ of $swk_{s,s'}$, i.e., $a = \langle \mathbf{a}, \mathbf{b} \rangle$. Then, it computes and returns $(a_0, a_1) = \lfloor P^{-1} \cdot \langle \mathbf{a}, swk_{s,s'} \rangle \rceil \bmod Q_\ell$.

### A.2. Addition module

This part introduces the function provided by full-RNS CKKS to achieve homomorphic addition between ciphertexts, which induces the underlying plaintexts to perform Hadamard addition.

**Plaintext Addition:** PAdd$(c, m)$ takes a plaintext polynomial $m \in R_{Q_\ell}$ and a ciphertext $c \in R_{Q_\ell}^2$ as input, both having the same scaling factor. It outputs a ciphertext $c' = c + (m, 0)$.

**Ciphertext Addition:** Add$(c_1, c_2)$ takes two ciphertexts $c_1, c_2 \in R_{Q_\ell}^2$ with the same scaling factor as input and outputs $c_1 + c_2$.

### A.3. Multiplication module

The multiplication module contains the functions to achieve homomorphic multiplication among ciphertexts and plaintexts, which induces the underlying plaintexts to perform the Hadamard product.

**Plaintext Multiplication:** PMult$(c, m)$: Given a plaintext polynomial $m \in R_{Q_\ell}$ and a ciphertext $c \in R_{Q_\ell}^2$ with scaling factors $\Delta$ and $\Delta'$ respectively, compute and return $c' = (c_0 \cdot m, c_1 \cdot m)$. The resulting ciphertext $c'$ has a scaling factor of $\Delta\Delta'$.

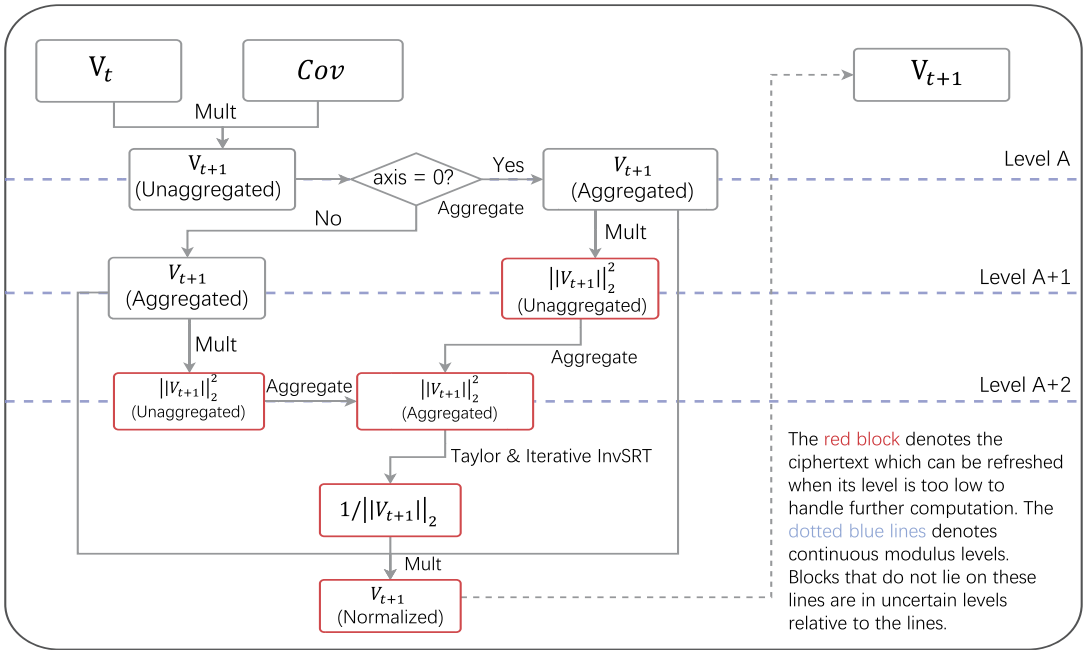


**Fig. 3.** Flow chart of the modulus refresh strategy in homomorphic PowerMethod.

**Ciphertext Multiplication:** Mult$(c_1, c_2)$: For ciphertexts $c$ and $c'$ with scaling factors $\Delta$ and $\Delta'$ respectively, compute $(d_0, d_1, d_2) = (c_0c'_0, c_0c'_1 + c_1c'_0, c_1c'_1)$ and return $d = (d_0, d_1) + \text{SwitchKey}(d_2, \text{rlk}) \in R^2_{Q_\ell}$ as the resulting ciphertext, where rlk is a switching key $swk_{s,s^2}$. The resulting ciphertext $d$ has a scaling factor of $\Delta\Delta'$.

A rescale operation is commonly used to transfer a ciphertext from a ring with a large modulus to a ring with a smaller modulus, reducing the expansive scaling factor in the ciphertext and limiting the error introduced by multiplications.

**Rescaling:** Rescale$(c)$: For a ciphertext $c \in R^2_{Q_\ell}$ with scaling factor $\Delta$, return $\lfloor q_\ell^{-1} \cdot c \rceil$. The resulting ciphertext has a scaling factor of $\Delta/q_\ell$.

**Bootstrapping:** Bootstrap$(c)$: Bring a ciphertext $c \in R^2_{Q_\ell}$ back to $R^2_{Q_{L-k}}$, where $k$ is the multiplication depth of the bootstrapping circuit.

### A.4. Rotation module

The rotation module includes the functions to achieve homomorphic position shifting in a ciphertext. Specifically, full-RNS CKKS allows us to homomorphically rotate the position of the components of the underlying plain complex vector in a ciphertext.

**Rotation:** Rot$(c_1, k)$: Given a ciphertext $c \in R^2_{Q_\ell}$ and a rotation key $\text{rtk}_k$, compute and output $(c_0^{5^k}, 0) + \text{SwitchKey}(c_1^{5^k}, \text{rot}_k)$, where rot is a switching key $swk_{s,s^{5^k}}$. This ciphertext represents the result of rotating all components of the message vector $\mathbf{z} \in \mathbb{C}^{N/2}$ encrypted by $c$ to the left by $i$ steps (or positions), denoted as $\mathbf{z}' = \rho(\mathbf{z}; i)$.

## Appendix B. Proofs for the vector normalization strategy

### B.1. Amplification upper bound of the shifted covariance

**Theorem 7.** *For the 1-shifted covariance variance matrix $\Sigma' = \Sigma - \lambda \mathbf{v}\mathbf{v}^T$ of the covariance matrix $\Sigma$, where $\mathbf{v}$ is the normalized approximate dominant eigenvector of $\Sigma$ obtained using the PowerMethod, and $\lambda = \frac{\langle \Sigma\mathbf{v}, \mathbf{v}\rangle}{\langle \mathbf{v}, \mathbf{v}\rangle}$, the matrices $\Sigma'$ and $\Sigma$ have the same scaling upper bound. In other words, the amplification effect of the transformation $\Sigma'$ on a vector does not exceed the amplification effect of the transformation $\Sigma$ on the same vector.*

**Proof.** Let the covariance matrix have $n$ eigenvectors: $x_1, x_2, \ldots, x_n$, corresponding to eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_n$. We only need to prove that $\lambda \le \lambda_1$. When $\mathbf{v}$ is equal to $\mathbf{x}_1$, the inequality is obviously true. When $\mathbf{v}$ is not equal to $\mathbf{x}_1$, it can be represented as $\mathbf{v} = c_1\mathbf{x}_1 + c_2\mathbf{x}_2 + \cdots + c_n\mathbf{x}_n$, where $c_1^2 + c_2^2 + \cdots + c_n^2 = 1$. In this case, $\langle \Sigma\mathbf{v}, \mathbf{v}\rangle = \lambda_1 c_1^2 + \lambda_2 c_2^2 + \cdots + \lambda_n c_n^2 \le \lambda_1(c_1^2 + \cdots + c_n^2) = \lambda_1$, while $\langle \mathbf{v}, \mathbf{v}\rangle$ is always equal to 1. Therefore, $\lambda \le \lambda_1$ holds true. □

### B.2. Proof of effectiveness for the PowerMethod simulation

**Theorem 8.** *If a specific instance of the Lazy Normalization strategy can maintain an error bound of $e$ under a PowerMethod simulation with a Contraction Coefficient of $S$ for $t$ iterations, then for any covariance matrix with eigenvalues all greater than $S$, this instance guarantees that the length of the output approximate eigenvector within $t$ PowerMethod iterations is close to 1 with an error no bigger than $e$.*

**Proof.** We need to prove that the scaling effect of the covariance matrix transformation on any vector $\mathbf{v}$ with a length of 1 is not smaller than its minimum eigenvalue $\lambda_n$. Let $\lambda_1, \lambda_2, \ldots, \lambda_n$ represent the eigenvalues of $\Sigma$ arranged in descending order, and $\mathbf{x}_1, \ldots, \mathbf{x}_n$ be their corresponding eigenvectors. It is known that $\mathbf{v}$ can be expressed as: $\mathbf{v} = c_1\mathbf{x}_1 + c_2\mathbf{x}_2 + \cdots + c_n\mathbf{x}_n$, where $c_1^2 + c_2^2 + \cdots + c_n^2 = 1$. Then $||\Sigma\mathbf{v}||_2^2 = \lambda_1^2 c_1^2 + \cdots + \lambda_n^2 c_n^2 \ge \lambda_n^2(c_1^2 + \cdots + c_n^2) = \lambda_n$. □

## Appendix C. Modulus refresh timing in the privacy-preserving PCA algorithm

No modulus refresh operations are necessary during the computation of the covariance matrix, as its multiplication depth can be handled easily within concrete CKKS schemes. Regarding the PowerMethod, the timing of modulus refresh operations is illustrated in Flowchart in Fig. 3. As depicted in the flowchart, we avoid performing modulus refresh during the covariance matrix transformation because doing so would increase the frequency of modulus refreshes proportional to the size of the covariance matrix. Instead, we only refresh some intermediate values or the approximate eigenvector. Specifically, we manage to perform modulus refresh only once when refreshing the approximate eigenvector. This is achieved by extracting the components of

the approximate eigenvector distributed across different ciphertexts and encoding them into a single ciphertext. After the refresh, the components can be re-distributed into different ciphertexts. This strategy effectively decouples the number of modulus refreshes from the number of vector components. In addition, it is worth noting that the cloud users in our scheme are not involved in any aspect of the privacy-preserving PCA algorithm, apart from encryption and decryption operations. Unlike previous approaches that require user participation in the privacy-preserving algorithmic process (Rathee et al., 2018), (Lu et al., 2016), our scheme not only safeguards the algorithm privacy of the cloud service provider but also minimizes the coupling between the privacy-preserving PCA algorithm and the design of the communication protocol between the cloud and the user.

## References


Arnold, David, Saniie, Jafar, 2023. Evaluation of homomorphic encryption for privacy in principal component analysis. In: 2023 IEEE International Conference on Electro Information Technology (eIT). IEEE, pp. 284–288.

Belhumeur, Peter N., Hespanha, Joao P., Kriegman, David J., 1997. Eigenfaces vs. fisherfaces: recognition using class specific linear projection. IEEE Trans. Pattern Anal. Mach. Intell. 19 (7), 711–720.

Bossuat, Jean-Philippe, Mouchet, Christian, Troncoso-Pastoriza, Juan, Hubaux, Jean-Pierre, 2021. Efficient bootstrapping for approximate homomorphic encryption with non-sparse keys. In: Advances in Cryptology–EUROCRYPT 2021: 40th Annual International Conference on the Theory and Applications of Cryptographic Techniques. Zagreb, Croatia, October 17–21, 2021. In: Proceedings, Part I. Springer, pp. 587–617.

Brakerski, Zvika, Gentry, Craig, Vaikuntanathan, Vinod, 2014. (leveled) fully homomorphic encryption without bootstrapping. ACM Trans. Comput. Theory 6 (3), 1–36.

Cheon, Jung Hee, Kim, Andrey, Kim, Miran, Song, Yongsoo, 2017. Homomorphic encryption for arithmetic of approximate numbers. In: Advances in Cryptology–ASIACRYPT 2017: 23rd International Conference on the Theory and Applications of Cryptology and Information Security. Hong Kong, China, December 3–7, 2017. In: Proceedings, Part I, vol. 23. Springer, pp. 409–437.

Cheon, Jung Hee, Han, Kyoohyung, Kim, Andrey, Kim, Miran, Song, Yongsoo, 2019. A full rns variant of approximate homomorphic encryption. In: Selected Areas in Cryptography–SAC 2018: 25th International Conference. Calgary, AB, Canada, August 15–17, 2018. In: Revised Selected Papers, vol. 25. Springer, pp. 347–368.

Deng, Li, 2012. The mnist database of handwritten digit images for machine learning research. IEEE Signal Process. Mag. 29 (6), 141–142.

Duong, Dung Hoang, Kumar Mishra, Pradeep, Yasuda, Masaya, 2016. Efficient secure matrix multiplication over lwe-based homomorphic encryption. Tatra Mt. Math. Publ. 67 (1), 69–83.

Fan, Junfeng, Vercauteren, Frederik, 2012. Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive.

Froelicher, David, Cho, Hyunghoon, Edupalli, Manaswitha, Sa Sousa, Joao, Bossuat, Jean-Philippe, Pyrgelis, Apostolos, Troncoso-Pastoriza, Juan R., Berger, Bonnie, Hubaux, Jean-Pierre, 2023. Scalable and privacy-preserving federated principal component analysis. arXiv preprint, arXiv:2304.00129.

Gizem, S. Cetin, Doroz, Yarkin, Sunar, Berk, Martin, William J., 2015. Arithmetic using word-wise homomorphic encryption. Cryptology ePrint Archive.

Halevi, Shai, Shoup, Victor, 2014. Algorithms in helib. In: Advances in Cryptology–CRYPTO 2014: 34th Annual Cryptology Conference. Santa Barbara, CA, USA, August 17–21, 2014. In: Proceedings, Part I, vol. 34. Springer, pp. 554–571.

Halevi, Shai, Shoup, Victor, 2018. Faster homomorphic linear transformations in helib. In: Advances in Cryptology–CRYPTO 2018: 38th Annual International Cryptology Conference. Santa Barbara, CA, USA, August 19–23, 2018. In: Proceedings, Part I, vol. 38. Springer, pp. 93–120.

Hotelling, Harold, 1933. Analysis of a complex of statistical variables into principal components. J. Educ. Psychol. 24 (6), 417.

Jang, Jaehee, Lee, Younho, Kim, Andrey, Na, Byunggook, Yhee, Donggeon, Lee, Byounghan, Hee Cheon, Jung, Yoon, Sungroh, 2022. Privacy-preserving deep sequential model with matrix homomorphic encryption. In: Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security, pp. 377–391.

Jiang, Xiaoqian, Kim, Miran, Lauter, Kristin, Song, Yongsoo, 2018. Secure outsourced matrix computation and application to neural networks. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, pp. 1209–1222.

Lattigo v4, Online: https://github.com/tuneinsight/lattigo. EPFL-LDS, Tune Insight SA.

Liu, Yingting, Chen, Chaochao, Zheng, Longfei, Wang, Li, Zhou, Jun, Liu, Guiquan, Yang, Shuang, 2020. Privacy preserving pca for multiparty modeling. arXiv preprint. arXiv:2002.02091.

Lu, Wen-jie, Kawasaki, Shohei, Sakuma, Jun, 2016. Using fully homomorphic encryption for statistical analysis of categorical, ordinal and numerical data. Cryptology ePrint Archive.

Lyubashevsky, Vadim, Peikert, Chris, Regev, Oded, 2010. On ideal lattices and learning with errors over rings. In: Advances in Cryptology–EUROCRYPT 2010: 29th Annual International Conference on the Theory and Applications of Cryptographic Techniques. French Riviera, May 30–June 3, 2010. In: Proceedings, vol. 29. Springer, pp. 1–23.

Ma, Xirong, 2023. Buffaloheextension. https://github.com/lilBuffaloEric/BuffaloHE_extension.

Mishra, Pradeep Kumar, Rathee, Deevashwer, Hoang Duong, Dung, Yasuda, Masaya, 2021. Fast secure matrix multiplications over ring-based homomorphic encryption. Inf. Secur. J. Glob. Perspect. 30 (4), 219–234.

Mouchet, Christian, Troncoso-Pastoriza, Juan, Bossuat, Jean-Philippe, Hubaux, Jean-Pierre, 2021. Multiparty homomorphic encryption from ring-learning-with-errors. In: Proceedings on Privacy Enhancing Technologies. 2021(CONF), pp. 291–311.

Panda, Samanvaya, 2021. Principal component analysis using ckks homomorphic scheme. In: Cyber Security Cryptography and Machine Learning: 5th International Symposium. CSCML 2021, Be'er Sheva, Israel, July 8–9, 2021. In: Proceedings, vol. 5. Springer, pp. 52–70.

Panda, Samanvaya, 2022. Polynomial approximation of inverse sqrt function for fhe. In: Cyber Security, Cryptology, and Machine Learning: 6th International Symposium, Proceedings. CSCML 2022, Be'er Sheva, Israel, June 30–July 1, 2022. Springer, pp. 366–376.

Pearson, Karl, 1901. Liii. on lines and planes of closest fit to systems of points in space. Lond. Edinb. Dublin Philos. Mag. J. Sci. 2 (11), 559–572.

Qu, Hongyuan, Xu, Guangwu. Improvements of homomorphic evaluation of inverse square root. Available at SSRN 4258571.

Rathee, Deevashwer, Kumar Mishra, Pradeep, Yasuda, Masaya, 2018. Faster pca and linear regression through hypercubes in helib. In: Proceedings of the 2018 Workshop on Privacy in the Electronic Society, pp. 42–53.

von Mises, Richard, Pollaczek-Geiringer, Hilda, 1929. Praktische verfahren der gleichungsauflösung. Z. Angew. Math. Mech. 9, 152–164.

Wang, Shufang, Huang, Hai, 2019. Secure outsourced computation of multiple matrix multiplication based on fully homomorphic encryption. KSII Trans. Int. Inf. Syst. 13 (11), 5616–5630.

Xiao, Han, Rasul, Kashif, Vollgraf, Roland, 2017. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. arXiv preprint. arXiv:1708.07747.



**Xirong Ma** is currently pursuing his bachelor's degree at the School of Software Engineering, Shandong University, starting from 2020. His research interests include privacy-preserving computation, homomorphic encryption, and machine learning.

**Chuan Ma** received the B.S. degree from the Beijing University of Posts and Telecommunications, Beijing, China, in 2013 and Ph.D. degree from the University of Sydney, Australia, in 2018. From 2018 to 2022, he worked as a lecturer in the Nanjing University of Science and Technology, and now he is a principal investigator at Zhejiang Lab. He has published more than 40 journal and conference papers, including a best paper in WCNC 2018, and a best paper award in IEEE Signal Processing Society 2022. His research interests include stochastic geometry, wireless caching networks and distributed machine learning, and now focuses on the big data analysis and privacy-preserving.

**Yali Jiang** is currently a Lecturer in the School of Software, Shandong University. She received her B.Sc., M.Sc., and Ph.D. degrees from Shandong University in 1999, 2002, and 2011, respectively. She is engaged in research in the field of information security and cryptography. Her main research areas focus on privacy-preserving technologies based on cryptography, including Fully Homomorphic Encryption (FHE), Secure Multi-Party Computation (MPC), Zero-Knowledge Proofs (ZKP), and more. She has participated in the National Key R&D Programs of China, the Shandong Provincial Key Research and Development Programs, the Shandong Provincial Natural Science Foundation, and joint research projects with enterprises.

**Chunpeng Ge** is currently a full professor at the School of Software, Shandong University. His current research interests include information security and privacy-preserving for cloud computing, blockchain, security and privacy of AI systems. He has published more than 60 papers in prestigious journal and conferences. He served as the editor of the journal of CSI and program committee for more than 30 international conferences.